FH | JOANNEUM
University of Applied Sciences

Automated Bug Bounty Hunting

Master's Thesis
submitted in conformity with the requirements for the degree of
Master of Science in Engineering (MSc)
Master's degree programme **IT & Mobile Security**

FH JOANNEUM (University of Applied Sciences), Kapfenberg

Supervisor: FH-Prof. DI Dr. Klaus Gebeshuber
Submitted by: Ing. Simon Schönegger, BSc
Personal identifier: 51810443

August 2022

Abstract

While recent trends show that software has become increasingly secure, it is still important to ensure its security. The practice of penetration testing is common in the software development field. Penetration testing is the procedure of effectively testing for vulnerabilities in the software and infrastructure of a company. In a normal setup, a small team is engaged to take part in a penetration test. Most penetration tests take place in a non-productive environment.

In contrast to penetration testing, bug bounty hunting does not limit the team taking part in the tests to a limited number of researchers. Bug bounty hunting is an approach to crowd source the actual penetration testing, allowing an unlimited number of security researchers to test applications for security purposes while offering monetary rewards. (Akgul, Eghtesad & Laszka, 2020, p. 1) Since these tests are not performed solely by hand, security researchers utilise various tools to find vulnerabilities in the target application. As tools are used within bug bounty hunting a researcher may utilise these tools one after another to find as many vulnerabilities as possible, rewarding them with rewards paid by the company offering the bug bounty hunting program. Companies may define different rules and requirements for researchers taking part in these tests.

Automation in the context of bug bounty hunting is a tempting idea since it can be utilised to acquire monetary rewards. (Walshe & Simpson, 2020, p. 41) Many researchers have attempted to automate toolchains, but there seems to be no one-fits-all solution for finding vulnerabilities across different programs. The aim of this thesis is to implement an automated approach capable of being used among different programs while maintaining a high rate of true positives and a low rate of false positives.

The results of this paper show that it is possible to implement a flexible toolchain which automates steps performed during bug bounty hunting. The prototype is capable of automatically scanning domains of a target while displaying results in a human-readable form and maintaining a high rate of true positives and a low rate of false positives. However, the automatic generation of vulnerability submissions in the context of bug bounty hunting is beyond the scope of this thesis.

This thesis should serve as a possible approach for automating bug bounty hunting and penetration testing. Recommendations on which tools could be used in such an automated approach and their benefits and downsides are thematised. This thesis also discusses the evolution of bug bounty hunting programs in Internet of Things (IoT).

Zusammenfassung

Momentane Entwicklungen in der Cyber-Security Branche zeigen, dass Software zunehmend sicherer gestaltet wird. Die Durchführung von Penetration-Tests ist mittlerweile eine bekannte Herangehensweise, um die Sicherheit von Software-Komponenten und Systemen zu gewährleisten. Penetration-Tests sind Sicherheitsüberprüfungen, welche durchgeführt werden um Schwachstellen in der Software und Infrastruktur von Unternehmen aufzudecken. Im Normalfall werden Penetration-Tests von einer kleinen Gruppe von Sicherheitsexperten durchgeführt. Solche Tests werden zumeist nicht in der Produktionsumgebung von Unternehmen durchgeführt.
Im Gegensatz zu Penetration-Testing wird bei Bug Bounty Hunting die Anzahl der Sicherheitsexperten nicht auf eine kleine Gruppe reduziert. Bug Bounty Hunting verfolgt die Herangehensweise jedem Sicherheitsexperten die Möglichkeit zu bieten die Sicherheit von verschiedenen Software-Komponenten und Systemen zu evaluieren. Während bei Penetration-Tests die Bezahlung auf Projektbasis erfolgt, werden Sicherheitsexperten bei Bug Bounty Hunting nach der Anzahl und Kritikalität der gefundenen Schwachstellen bezahlt. Zumeist werden in Bug Bounty Hunting verschiedene Werkzeuge verwendet um Schwachstellen aufzudecken. Diese Werkzeuge werden von Sicherheitsexperten zumeist manuell nacheinander eingesetzt, um so viele Schwachstellen wie möglich zu finden. Im Bug Bounty Hunting werden von Firmen verschiedene Regeln und Voraussetzungen definiert, an welche sich die teilnehmenden Experten zu halten haben.
Mit der steigenden Popularität von Bug Bounty Hunting wuchs auch die Nachfrage nach automatisierten Lösungen, um Sicherheitstests durchzuführen, da Automatisierung in diesem Kontext zu erhöhter, passiver Bezahlung führen kann. Viele Sicherheitsexperten versuchten bereits solche Sicherheitsüberprüfungen zu automatisieren aber, es scheint keine Lösung zu geben, die flexibel genug ist um in einer Vielzahl von Bug Bounty Hunting Programmen angewandt zu werden. Das Ziel dieser Arbeit ist es, eine automatisierte Lösung für Sicherheitsüberprüfungen zu entwickeln, welche in verschiedenen Bug Bounty Hunting Programmen zum Einsatz kommen kann. Diese automatisierte Lösung soll ebenfalls eine hohe Rate an tatsächlichen Schwachstellen aufweisen und gefundene false positives auf ein Mindestmaß reduzieren.
Die Resultate zeigen, dass die implementierte automatisierte Lösung es ermöglicht, verschiedene Schritte im Bug Bounty Hunting zu automatisieren. Der implementierte Prototyp ermöglicht es verschiedene Domains des Ziels automatisch zu enumerieren und auf Schwachstellen zu testen. Die gefundenen Resultate werden in einer Form dargestellt, welche für Menschen leicht interpretierbar ist. Zusätzlich reduziert der Prototyp die gefundenen false positives auf ein Minimum. Die automatische Generierung von Schwachstellen-Meldungen wird nicht als Ziel dieses Prototypen betrachtet.
Diese Arbeit soll als Grundlage für Sicherheitsexperten dienen, welche sich das Ziel gesetzt haben Bug Bounty Hunting oder Penetration-Tests zu automatisieren. Es werden Empfehlungen zu verschiedenen Werkzeugen abgegeben und die Vor- und Nachteile dieser Werkzeuge aufgezeigt. Diese Arbeit beschäftigt sich ebenfalls mit Bug Bounty Hunting im Kontext von IoT.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Currently, there are few automated approaches available for automating bug bounty hunting. As Table 1.1 depicts, the interest in bug bounty programs drastically increased in recent years. The goal of this thesis is to implement a flexible toolchain composing various free tools into a fixed workflow. Such workflow should be able to discover as many vulnerabilities as possible while maintaining a low rate of false positives. This toolchain shall be tested in different bug bounty programs of different platforms.

Year	**New programs**
2013-2014	11
2014-2015	26
2015-2016	33
2016-2017	37
2017-2018	43
2018-2019	50

Table 1.1: Number of new bug bounty programs on hackerone.com according to Walshe & Simpson, 2020, p. 36

1.1 Problem Statement

As Hakluke, 2021 suggests, many researchers employ automation in bug bounty hunting. However, few automated approaches are publicly available. However, a plethora of tools for various tasks are publicly available. It is debatable how these different tools may be composed into a flexible toolchain automating the bug bounty hunting process. This thesis aims to integrate many tools into a toolchain and to yield accurate results.

1.2 Research Questions

The aim of this thesis is to implement a toolchain suitable for bug bounty hunting in different bug bounty hunting programs. It should serve as a flexible approach to automate this process while still maintaining accurate results.
The main research questions of this thesis are as follows:

- How precise is an automated toolchain in the context of bug bounty hunting?
- Which types of vulnerabilities may be detected?
- Can an automated toolchain, composed mainly of free tools compete in bug bounty hunting?
- What are the requirements for the toolchain in order to qualify for bug bounty hunting programs?
- What cutting-edge technology has an impact on bug bounty hunting?
- What Niches in bug bounty hunting are not yet covered?

1.3 Hypothesis

As the popularity of bug bounty hunting programs increased, researchers implemented different approaches to automate the discovery of vulnerabilities in the context of bug bounty hunting. The use of automation increases productivity in bug bounty hunting and also increases pay-outs received by the researchers. However, only a few automated bug bounty hunting implementations are publicly available. Therefore, this thesis aims to implement a toolchain capable of increasing the productivity as well as the pay-outs in bug bounty hunting. To prove the sufficiency of such implementation, different experimental approaches are performed. The first step is to create an automated toolchain consisting of widely used tools. After the creation of the prototype, a

series of experiments will happen. The tools used in the toolchain are based on individual experience as well as the Open Web Application Security Project (OWASP) Top 10 Web Application Security Risks.[1] The toolchain is used in different bug bounty hunting programs to measure its efficiency. The bug bounty programs are not taken from a single platform. The following platforms are used in the experiments:

- BugCrowd[2]
- HackerOne[3]
- YesWeHack[4]
- Intigriti[5]

When the toolchain is running in different bug bounty hunting programs, a constant refinement of the prototype is required. The toolchain must adapt to different conditions and fulfil different requirements based on the individual requirements of the program.
In regard to the technology stack, the toolchain is meant to consist of three main parts. A universal server backend is responsible for interacting with the integrated tools. NodeJS[6] is chosen as the backend technology. The backend will interact with a database and persist the results of different tools. This makes the toolchain less invasive to previously scanned targets since it is possible to fetch results from a database rather than running tests against an actual target.
PostgreSQL[7] is chosen as the database management system. The toolchain also requires a client application responsible for displaying the results and composing a toolchain by interacting with the backend in a uniform way.
As for the client technology, an Angular[8] web application is used.
In contrast to the approach of using a web-based frontend, another python based client is implemented. This client is used to conduct long-running scans which do not require intermediate user interaction. The results of this client are rendered in Hypertext Markup Language (HTML) format. This client implementation also utilises the NodeJS server backend required for the invocation of different tools.
The automated reporting of vulnerabilities is beyond the scope of this thesis. However, the results are displayed in a client along with the location of the finding and the

[1] https://owasp.org/www-project-top-ten/
[2] https://bugcrowd.com/
[3] https://www.hackerone.com/
[4] https://yeswehack.com/
[5] https://www.intigriti.com/
[6] https://nodejs.org/en/
[7] https://www.postgresql.org/
[8] https://angular.io/

payload used to find it. With this information, the end user of the toolchain is able to verify the findings on their own and report them if they prove to be valid. Techniques reducing the amount of false positives should be implemented to report findings to the end user which have a high chance of being valid.
With the fulfilment of these requirements, a researcher may be able to improve the productivity and pay-out rate with the use of this automated approach since the results are stored in a human-readable manner and the rate of false positives is decreased.

1.4 Method

As mentioned previously, this thesis includes a series of experiments. Once the implementation of the toolchain is complete, it will be tested in bug bounty hunting programs on various platforms to evaluate its feasibility. The feasibility of the prototype is measured using the following criteria:

- Resource consumption
- Caused traffic
- Performance in bug bounty hunting programs

The performance of the prototype in bug bounty hunting programs is measured based on the validity of the results. A large number of false positives delivered by the prototype therefore degenerates the overall performance. Results considered valid by the bug bounty hunting platform increase the measured performance of the prototype.
Based on these criteria, it is possible to answer the research questions as well as to determine the actual feasibility of the prototype in terms of expected costs.

Chapter 2

Related Work

2.1 Introduction to Bug Bounty Hunting

To provide insight into the practice of bug bounty hunting, it is necessary to understand the fundamentals of bug bounty hunting. As the name indicates, bug bounty hunting focuses on the discovery of bugs. A bug is a flaw in a software component which proposes the risk of a system being compromised in some manner. The term 'bug' in this thesis does not reference simple software bugs (i.e. errors in a formula) since it rather focuses on security relevant issues which put a system or application at risk.
Traditionally, organisations have relied on the work of internal security experts and outsourced experts (e.g. external penetration testing) to identify security relevant bugs. (Akgul, Eghtesad & Laszka, 2020, p. 1) In contrast, the approach of bug bounty hunting crowdsources the discovery of security relevant bugs to external security experts who are not employed or contracted by the organisation itself. (ibid., p. 1) To motivate external security researchers to search for bugs in the product of the organisation, organisations offer rewards based on the severity of the discovered vulnerability. These rewards range from monetary rewards to acknowledgements by the organisation. (ibid., p. 1) Bug bounty hunting differs from the well-known practice of vulnerability disclosure. Bug bounty hunting can be described as a kind of vulnerability disclosure since individuals participating in bug bounty programs are required to disclose the discovered vulnerabilities in a responsible manner. However, the traditional approach of vulnerability disclosure would not offer any rewards to the researcher in contrast to bug bounty hunting disclosure principles. (ibid., p. 1)
According to Fryer & Simperl, 2019, p. 1 bug bounty programs were first introduced by Netscape in the 1990s. Netscape started to offer monetary rewards for individu-

als finding security flaws in the Netscape browser. According to Kuehn & Mueller, 2014, p. 4, Netscape offered rewards ranging to $1000. Although the history of bug bounty hunting dates back to the 1990s, the examples of companies offering monetary rewards and a safe harbour for security researchers have been sparse until recently. (Fryer & Simperl, 2019, p. 1) Data from 2004-2005 show that researchers were not able to responsibly disclose vulnerabilities while gaining monetary rewards. (Kuehn & Mueller, 2014, p. 4) Black markets therefore formed where researchers could sell vulnerabilities and threat actors would buy them. This formation of black markets contradicts the principle of responsible disclosure and eliminates all ethics associated with vulnerability research. Many years had to pass until the establishment of bug bounty programs would offer an ethical way to disclose vulnerabilities while still gaining monetary rewards. Bug bounty programs have become increasingly popular since major companies have started to offer bug bounty programs. With the increased amount of bug bounty hunting programs, this approach has become increasingly popular in the process of vulnerability management. Bug bounty hunting has even been integrated in the secure software development lifecycle frameworks to help security teams to properly address release and maintenance phases. (Walshe & Simpson, 2020, p. 35) Major companies offer bug bounty hunting on their own. An example of this are Google[1], Microsoft[2] and Mozilla[3]. Smaller companies typically do not offer bug bounty hunting programs on their own but rather move to a bug bounty hunting platform like HackerOne[4]. Such bug bounty hunting platforms offer both free and paid services to organisations wishing to run a bug bounty hunting program. (ibid., p. 35) Table 1.1 offers an overview of the increasing popularity of bug bounty hunting programs on HackerOne. Bug bounty hunting has become increasingly popular for small and large companies on this platform since 2015. In 2013, BugCrowd, a bug bounty hunting platform similar to HackerOne, was established. According to Subramanian & Malladi, 2020, p. 39 in 2013 a private intermediary received 100 reports whereas this number rose to 700 received reports in 2017. This outlines the spike of interest in bug bounty hunting by the community.

Walshe & Simpson, 2020, p. 36 note that the sheer amount of security researchers participating in a bug bounty hunting program may lead to the elimination of most security vulnerabilities. Since bug bounty programs focus on rewarding security experts for their participation, they lead to fewer security experts selling information about vulnerabilities. Fryer & Simperl, 2019, p.2 found that individuals searching for secu-

[1]https://bughunters.google.com/
[2]https://www.microsoft.com/en-us/msrc/bounty
[3]https://www.mozilla.org/en-US/security/bug-bounty/
[4]https://www.hackerone.com/

rity vulnerabilities were tempted to sell information about discovered vulnerabilities on a black market, which offered monetary rewards for the discovered security vulnerabilities. However, the sold vulnerability would not be disclosed to the organisation, but rather be used by threat actors to compromise systems. Aside from threat actors state actors may have interests in buying information about vulnerability of systems, as well. (Fryer & Simperl, 2019, p. 2) Information on vulnerabilities in systems of a foreign state may be of great interest for national state actors. Such vulnerabilities may be used on different occasions (e.g. espionage and cyber warfare). Since state actors typically have a larger amount in both technical and knowledge-based resources than normal threat actors, the exploitation of the vulnerability may occur in a manner that could not be easily discovered by the organisation or targeted state.
With this information in mind, it is still questionable whether bug bounty hunting serves a social good. Vulnerabilities disclosed to the bug bounty hunting platform or organisation may eliminate the risk of a threat actor exploiting it. This implies that the vulnerability is addressed in a timely fashion and is patched up front. Black markets for vulnerabilities rely on the principle of supply and demand. If a vulnerability is put up for sale, a buyer has yet to be found and the price has yet to be agreed to. According to ibid., p. 2 there were several occasions where a vulnerability was put up for sale but was patched prior to being sold. Black market prices tend to be higher compared to pay-outs on bug bounty hunting platforms. The concept of bug bounty hunting does not eliminate black markets. Individuals with malevolent intentions may still sell vulnerabilities on black markets to achieve a higher monetary reward from a thread or state actor rather than responsibly disclosing it to the vendor using a bug bounty platform. Kuehn & Mueller, 2014, p. 6 describe the obstacles in a vulnerability market in the following manner:

- *"Vulnerability information is time sensitive"*: The value of a vulnerability may decrease over time due to patches or newer versions of software which eliminate a vulnerability. The vulnerability therefore may not be exploited to its full potential which leads to a decrease in the price.
- *"No transparency in pricing"*: No pricing information is publicly available. Prices are negotiated between buyer and seller. Prices of a vulnerability of a high severity may be lower compared to vulnerability of the same severity based on the technology used and the affected user base.
- *"Difficulty in finding buyers and sellers"*: No mechanism is in place to match buyers and sellers. The selling of a vulnerability may be a time-consuming procedure. Since the price of a vulnerability may decrease over time, sellers are

likely unable to sell the vulnerability for the desired price.

- *"Checking the buyer"*: Mostly vulnerabilities are sold to threat actors as indicated by Fryer & Simperl, 2019, p. 2 A seller with appropriate ethics would have to verify the buyer's planned use of the sold vulnerability. Finding a buyer with appropriate ethics can be a time-consuming process.
- *"Value cannot be demonstrated without loss"*: A seller is likely to have to demonstrate the discovered vulnerability to the buyer. The proper balance between showing the impact without showing the data that the buyer is interested in therefore must be found. If a seller demonstrates the vulnerability to the buyer and shows the data the buyer is interested in, the buyer may refrain from buying the vulnerability, as they already gathered the desired information. If the seller demonstrates the vulnerability in a manner which does not show sufficient data, the buyer may refrain from buying the vulnerability since they may not consider the vulnerability to be as impactful as desired.
- *"Ensuring claim to vulnerability"*: When a vulnerability is sold, the seller has no way to ensure the claim of the sold vulnerability. An individual may buy the vulnerability and sell the vulnerability to another party at a higher price. The buyer could easily claim the vulnerability as their own.
- *"Exclusivity of rights"*: The seller must agree to sell the vulnerability exclusively to the buyer to achieve the highest possible price. Since vulnerabilities are mostly an informational good the seller will know about all technical aspects of a certain vulnerability after the initial sale. This may cause the seller to sell the vulnerabilities to other buyers. The initial buyer would not have a chance to eliminate this risk.

The process of managed bug bounty hunting addresses most of these obstacles. The *time sensitivity of a vulnerability* becomes irrelevant since the researchers are working in a fixed scope. When a vulnerability occurs within the scope, it can be disclosed no matter how old the software is or how old the vulnerability is. Since the monetary reward is based on the severity of the vulnerability, the age of the vulnerability does not affect the price. The *lack of price transparency* is tackled since bug bounty hunting programs list the prices for vulnerabilities based on their severity. The severity is typically calculated using the Common Vulnerability Scoring System (CVSS) score. A managed bug bounty hunting program has no *difficulties in finding buyers and sellers*. Security experts act as the sellers and the organisation behind the bug bounty program acts as the buyer. The process of disclosing a vulnerability is not time-consuming in

such a set-up since prepared mechanisms are in place to disclose a vulnerability to the vendor. Once a vulnerability is disclosed and verified by the vendor, a pay-out must occur. *Verifying the buyer* is unnecessary in a bug bounty hunting program since the buyer is the vendor or an organisation (i.e. a trusted party). The *Value cannot be demonstrated without loss* obstacle does not affect managed bug bounty hunting Programs. A researcher must demonstrate the impact of a vulnerability in the disclosure process. However, the details disclosed in the submission do not affect the price of the vulnerability. The prices of vulnerabilities are driven by their severity. The vendor therefore does not refrain from "buying" the vulnerability if too much data of interest is disclosed. However, many bug bounty programs refrain from a complete compromise of the system. For critical vulnerabilities, a minimal Proof of Concept (PoC) is likely to be sufficient. Since bug bounty programs map the disclosed vulnerabilities to a researcher, there is no need to *ensure the claim to a vulnerability*. Vulnerabilities disclosed in such programs are not disclosed to the public unless they are fixed and both the vendor and the researcher agree to publicly disclose it. Many bug bounty programs have a procedure for duplicate submissions in place. However, the *exclusivity of rights* can prove to be a problem since the researcher has extended knowledge about the vulnerability. There is no guarantee that a researcher with malicious intentions will refrain from exploiting the vulnerability on their own or selling it on the black market. However, the rules of engagement are likely to include such behaviour and put legal actions in place if such scenario should occur.

The disclosure of vulnerabilities in a bug bounty hunting program relies on the concept of responsible disclosure. The first step of responsible disclosure is direct contact with the vendor disclosing limited details of the discovered vulnerability. This must happen in a way to confirm that the vendor has received details about the disclosed vulnerability. The use of a third-party coordinator may assist in this. However, the circle of trust should be kept small. (Shepherd, 2021, p. 9) After the initial disclosure, a deadline for the vendors' response should be placed. The generally accepted deadline is 30 days. (ibid., p. 9) After the initial disclosure, continued communication should be established. It is important for a vendor to reproduce the mentioned vulnerability and the researcher should provide all necessary information to the vendor. (ibid., p. 10) If the vendor does not respond, the researcher can publicly disclose the vulnerability. (ibid., p. 10) However, if the vendor is responsive and can reproduce the vulnerability, the next stage is patch development. In this stage the vendor is responsible for creating a patch which addresses the vulnerability. If additional products are affected by the vulnerability, the circle of trust should be widened to effectively mitigate the vulnerability in other products. (ibid., p. 10) Once a patch is delivered, the vendor and the researcher

should test its efficiency. Once the patch is proven to be effective, public disclosure is the next step. The researcher and the vendor must collaborate to file the content of the public disclosure. This content typically does not contain the exploit code. (Shepherd, 2021, p. 11) Bug bounty programs hosted on platforms such as HackerOne manage responsible disclosure with mechanisms in place for vulnerability reporting.

Altough the concept of responsible disclosure is widely accepted, there are several examples where this disclosure process did not work as expected. For instance, Lilith Wittmann responsibly disclosed a vulnerability in a German political party's survey application which questioned individuals about their experiences with the ruling political parties in Germany. (Corfield, 2022) The political party insisted that data gathered by their application were anonymised, however Wittmann discovered that this data were not anonymised. (ibid.) Wittmann informed the political party about the discovered issues in a responsible manner, but, no agreements regarding the disclosure of the vulnerability were made between Wittmann and the political party. (ibid.). She therefore made a public disclosure. The political party shut down the application. However, a few days later the police contacted Wittmann because the political party had accused her with data theft. (ibid.) The allegations against Wittmann were ultimately dropped. (ibid.) This example describes the problems with responsible disclosure without the usage of a bug bounty program in place, without a pre-defined set of rules of engagement, the reporter may face various allegations.

Host	Company	Payment Range
Apple	Apple	$0 - $1,000,000 (Apple, 2022)
HackerOne	Yahoo	$0 - $15,000 (Yahoo, 2022)
BugCrowd	Western Union	$100 - $3,000 (Union, 2022)
BugCrowd	Tesla	$100 - $15,000 (Tesla, 2022)
GitHub	GitHub	$617 - $30,000+ (GitHub, 2022)
Microsoft	Microsoft	$0 - $250,000 (Microsoft, 2022)
HackerOne	PayPal	$0 - $20,000 (PayPal, 2022)

Table 2.1: Monetary reward ranges of different providers

2.1.1 Taxonomy of Bug Bounty Hunting

As mentioned previously bug bounty hunting programs offer monetary rewards for researchers who find vulnerabilities based on the severity of the discovered vulnerability. Certain types of vulnerabilities are typically considered out of scope and are not rewarded in any way.

2.1.1.1 Pay-outs and In-scope Vulnerabilities

Depending on the form of the bug bounty hunting program, vulnerabilities considered in-scope may result in a pay-out to the researcher. Not all bug bounty hunting programs offer monetary rewards. However, an acknowledgement is presented to the researcher in most cases. Since the price range of vulnerability is determined by its severity, vulnerabilities with higher severity receive a larger monetary reward than vulnerabilities with low severity. Table 2.1 serves as an example of payment ranges of large companies with bug bounty programs. Companies with a large user base tend to extend their maximum bounties. For example, Apple offers a maximum pay-out of one million dollars for zero-day-zero-click vulnerabilities. (Apple, 2022) Vulnerabilities in programs with a large user base tend to pay higher bounties since they are keen on keeping their user base secure and a zero-day exploit discovered in such program could prove to be fatal for all users and degrade the company's reputation. According to Kuehn & Mueller, 2014, p. 6, there is a black market for such vulnerabilities, but the amount paid for such vulnerabilities is undisclosed.

2.1.2 Comparison of Bug Bounty Hunting Platforms

Figure 2.1 depicts the number of reports alongside their pay range for 54 vendors over two years as researched by Subramanian & Malladi, 2020, p. 46. Most vulnerabilities have a low severity based on the pay-outs. However, for vulnerabilities with higher

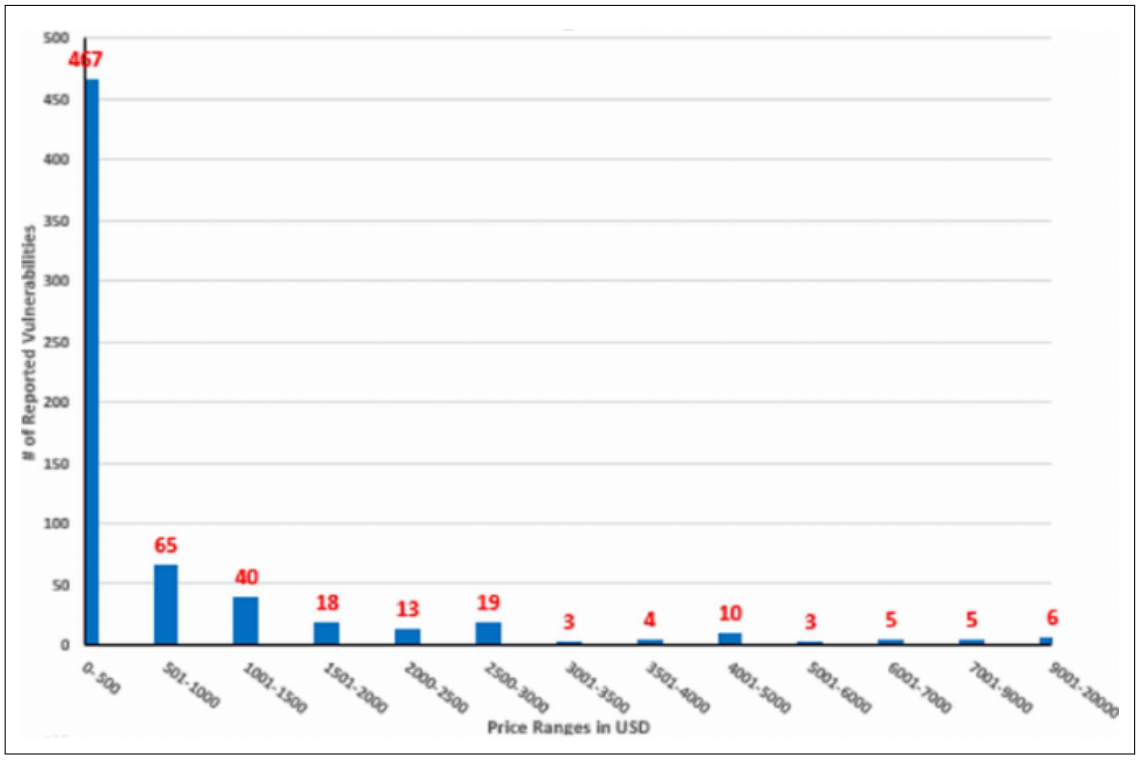

Figure 2.1: Vulnerability pay-out ranges based on 54 different vendors on HackerOne as researched by Subramanian & Malladi, 2020, p. 47

impact, reaching a price range between $3,500 and $3,000 is not uncommon. Vulnerabilities with a business critical impact and a price range between $6,000 and $20,000 are rarely discovered.

Table 2.2 depicts the most found vulnerabilities of 2020 as mentioned by HackerOne, 2022b. Cross Site Scripting (XSS) is the most rewarded vulnerability. Typically, XSS related vulnerabilities have a severity rating of low or medium. Therefore, it can be concluded that XSS based vulnerabilities are the most disclosed vulnerabilities. SQL injection (SQLi) vulnerabilities typically have a high or critical impact rating and are rewarded with over one million dollars in total. It is thus concluded, that SQLi vulnerabilities are the most common vulnerabilities with a high or critical impact rating. Code injection vulnerabilities, like SQLi vulnerabilities, are typically rated with a high or critical severity. However, code injections are rewarded by over a million dollars less compared to SQLi vulnerabilities. Code injection vulnerabilities are therefore likely to occur on rare occasions. Table 2.3 depicts that the most pay-outs are made for vulnerabilities with medium and low severity. Critical vulnerabilities seldom tend to appear.

Weakness Type	Total rewards
XSS	$4,211,006
Improper Access-Control	$4,013,316
Information Disclosure	$3,520,801
Server Side Request Forgery (SSRF)	$2,995,755
Insecure Direct Object Reference (IDOR)	$2,264,833
Privilege Escalation	$2,017,592
SQLi	$1,437,341
Improper Authentication	$1,371,863
Code Injection	$982,247
Cross Site Request Forgery (CSRF)	$662,751

Table 2.2: Top ten vulnerabilities rewarded as described by HackerOne, 2022b.

Severity	Proportion (%)	Average Cost
Critical	8.5	7,924$
High	19.6	2,610$
Medium	33.5	786$
Low	28.8	225$
None	9.6	0

Table 2.3: Latest 3000 reports submitted on HackerOne alongside their average cost and proportion as listed by Walshe & Simpson, 2020, p. 4

2.1.2.1 Out-of-scope Vulnerabilities and Approaches

Based on 1,028 publicly disclosed reports on HackerOne, Shafigh et al., 2021 developed an out-of-scope taxonomy model. According to ibid., p. 3, most reports were considered out-of-scope for the following reasons:

- Out-of-scope: A vulnerability was discovered in an application which was not listed as in scope or was explicitly listed as out-of-scope.
- False positive: The vulnerability is either an expected behaviour, is not exploitable, or is not security relevant.
- No PoC: No example was given on how the vulnerability may get exploited.
- Duplicated: The vulnerability was already found by another researcher.

Aside from these general reasons for disclosing reports as invalid, vulnerability- and procedure-based requirements may define a vulnerability as out-of-scope. According to an analysis by ibid., p. 3-5, vulnerabilities may be out-of-scope if they fulfil a certain criteria or were discovered using an approach considered invalid or unwanted by the bug bounty hunting program. Figure 2.2 depicts the most common reasons for a vulnerability to be considered out-of-scope. The term *distribution* refers to the

Category	Description	Attribute	Type	Distribution
Attack Surface	Any attack surfaces that is not in the interest or scope of a program	Website	contextual	54.8%
		3rd Party Services	contextual	45.2%
		Known Vulnerable Libraries	non contextual	25%
		Outdated Version of Services	non contextual	8.5%
		Testing/Prototyping environment	non contextual	2.7%
Vulnerability Type	Any vulnerability types that are not in the interest or has low security impact or would cause disruption on running services	Denial of Service	non contextual	73.4%
		Social Engineering	non contextual	69.7%
		Login/Logout CSRF	non contextual	49.5%
		Clickjacking	non contextual	47.9%
		Content Spoofing	non contextual	42.6%
		Spamming	non contextual	33%
		Fingerprinting	non contextual	31.9%
		Self XSS	non contextual	30.3%
		User Enumeration	non contextual	24.5%
Disclosure	Any information disclosure that is not in the interest or has low/non security impact	Error/Stack Traces Disclosure	non contextual	9.6%
		Public File/Info Disclosure	non contextual	9%
Configuration	Any miss or lack of configuration that leads to a low or non security impact	HTTP Configurations	non contextual	34.6%
		TLS/SSL Configurations	non contextual	34%
		SPF/DMARCK/DKIM Configurations	non contextual	26.1%
		Cookie Flags Configurations	non contextual	23.9%
		Password/Captcha Configurations	non contextual	22.9%
		Rate Limiting Configurations	non contextual	14.9%
Sec-pro Environment	Any settings of the environment that a sec-pro conduct their discovery task that is not compliant with the minimum expectation of the policy of a program	Outdated Running Environment	non contextual	19.1%
		Browser Autocomplete	non contextual	10.6%
Sec-pro Action	Any sec-pro activities that are prohibited by the policy of a program	Physical Attack	non contextual	68.6%
		Attack/Test End User	non contextual	27.7%
		Collecting/Modifying Sensitive Data	non contextual	13.8%
		Attack Infrastructure or Server	non contextual	9%
		Sharing Access or Vulnerability	non contextual	5.3%
		Malicious File Upload	non contextual	2.7%
Ad-hoc	Any ad-hoc criteria defined in the policy of a program	Existence of a keyword	contextual	1.6%

Figure 2.2: Out-of-scope taxonomy model as described by Shafigh et al., 2021, p. 5

number of bug bounty hunting programs which consider the approach of research or the vulnerability itself as out-of-scope. The data are based upon an analysis of over 320 public bug bounty hunting programs conducted by Shafigh et al., 2021.

2.1.3 Comparison of Bug Bounty Hunting Platforms

In order to compare bug bounty bunting platforms efficiently, it is necessary to identify the differences between platform intermediaries, bug brokers and corporate bug bounty programs and highlight the requirements for a bug bounty platform to be successful. In contrast to vulnerability black markets, bug bounty hunting platforms must provide an environment where information regarding vulnerabilities can be transferred between the researcher and vendor in a reliable and fair manner. It is therefore necessary for the researcher to include all information about a certain vulnerability while submitting the report. Reports must be confidential. Only the vendor, the researcher, and in

some cases representatives of the bug bounty hunting platform shall therefore have access to undisclosed vulnerabilities. The more information the researcher transfers, the higher the chances for a valid submission which lowers the transaction cost between researcher and vendor. (Subramanian & Malladi, 2020, p. 44) Another requirement for a successful bug bounty hunting platform is the transaction fairness of the vendor. Vendors must pay bounties once the vulnerability is disclosed by the researcher and accepted, which also means that the researcher must submit the vulnerability in a way which is easily understandable by the vendor. (ibid., p. 44) It is also required to address the fungibility of vulnerabilities. A researcher must convert discoveries into an appropriate financial reward in line with the expectation of the vendors and intermediates. (ibid., p. 44) In certain cases vendors may distance themselves from rewarding vulnerabilities in a monetary way. It is common that the vendor or bug bounty hunting platform offers rewards in form of reputation scores.

Bug bounty platforms can be divided into three different types: platform intermediaries, bug brokers and corporate bug bounty platforms.

Platform intermediaries are platforms between researcher and vendor (e.g. HackerOne and BugCrowd) and simultaneously offer various channels for bug bounty programs. Researchers utilise such channels to report vulnerabilities to the vendor. Platform intermediaries often employ their own staff responsible for triaging received vulnerabilities. Triage therefore often happens faster compared to other approaches such as corporate bug bounty programs. Once the vulnerability has been triaged and is considered valid the vendor joins the disclosure process. Using this procedure, the cost for the vendor is lowered since the initial triage is performed by the intermediary. (ibid., p.45) Such platforms offer a fair environment for both researcher and vendor since all transactions are supervised by the intermediary. A vendor therefore cannot deny payment if the vulnerability has been triaged by the intermediary since intermediaries typically have contracts with the vendor which do not allow such behaviour.

Bug brokers are platforms or individuals who buy vulnerabilities to sell them back to the vendor (e.g. Zerodium and Sonosoft). On such platforms, the researcher sells the discovered vulnerability to the platform rather than to the vendor directly. The difference between such platforms and platform intermediaries is that the transaction and information exchange between researcher and vendor occurs on behalf of the platform intermediary, but active exchange happens between researcher and vendor. Bug brokers do not follow this concept. No direct exchange happens between researcher and vendor since the researcher sells the vulnerability to the broker, and the broker sells the vulnerability to the vendor. Monetary rewards may often be higher than rewards on platform intermediaries or corporate bug bounty platforms. (ibid., p. 45) However,

monetary rewards are not necessarily fair for the researcher since the broker may sell the vulnerability back to the vendor for a higher price. (Subramanian & Malladi, 2020, p.45) Additionally, the selling of vulnerabilities depends on the liquidity in markets.

Corporate bug bounty programs are run by a certain company on private terms (e.g. Microsoft, Facebook and Apple). The vendor set policies and rules of engagement. The disclosure and information exchange occurs directly between the vendor and the researcher - no intermediary is placed between vendor and researcher. The vendor is therefore also accountable for the triage of vulnerabilities. The bounties paid may differ based on the focus area. (Microsoft, 2022) However, since the researcher and vendor are in direct contact, the researcher has the risk of being exposed when using techniques which are not compliant with the vendor's policies. Since the vendor is responsible for the triage, it is necessary to deal with duplicate reports, false positives, and spam which are common in such programs. Subramanian & Malladi, 2020, p. 45 estimates that up to 75% of reports can be considered spam in such engagements. The transaction fairness depends on the vendor. If the vendor is not willing to pay, the researcher cannot intervene since no intermediary supervisor is in place. However, such programs offer the potential to expand contracts for additional support. (ibid., p. 45)

In the context of this thesis, the main focus relies on platform intermediaries and corporate bug bounty programs. Since Corporate bug bounty programs cannot be reliably compared due to the lack of publicly available data and the differences in the technology stack, platform intermediaries will be compared based on the public available data.

The two most well-known bug bounty platforms are HackerOne and BugCrowd. These platforms offer publicly available datasets which are useful to initiate a comparison between the two platforms. As of 2022, HackerOne has 398 bug bounty programs publicly available. (HackerOne, 2022a) In comparison, BugCrowd has 317 bug bounty programs publicly available. (BugCrowd, 2022) While both platforms have a similar amount of publicly available programs, the number of private programs hosted on these platforms is unknown since these programs are only available to researchers who have been invited to join such a program. In terms of the number of researchers actively submitting vulnerabilities, HackerOne state that over 4,000 researchers actively submit vulnerabilities which, is an increase of 63% between 2019 and 2020.(HackerOne, 2019) In contrast, BugCrowd claims to have 3,000 active researchers. (Sharma, 2013) The data regarding the number of researchers are from 2013 and no new data have been published since. The active number on researchers on BugCrowd therefore remains unknown for 2022. With the known data, the ratio between programs and researchers would be 0.095 for HackerOne and 0.105 for BugCrowd. Fewer researchers

thus conduct research on programs published on HackerOne. However, this data must be taken with a grain of salt since the actual number of researchers on BugCrowd is unknown for 2022. However, fewer researchers conduct research on programs published on HackerOne than on BugCrowd since the number of researchers is likely to have increased between 2013 and 2022.
Table 2.4 depicts the percentage of reports considered valid or duplicate as stated by Zhao, Laszka & Grossklags, 2017, p. 376. As depicted, more duplicate reports are received on BugCrowd than on HackerOne. Based on this data, 62% of reports on HackerOne are considered invalid and 45.27% of reports on BugCrowd are considered invalid. The triage of HackerOne may thus be more strict compared to the triage of BugCrowd since HackerOne considers 14.73% more reports to be invalid than BugCrowd. However, the number of valid reports is 6.5% lower on BugCrowd compared to HackerOne. Researchers are therefore more likely receive a pay-out on HackerOne than on BugCrowd. ibid. cooperated with HackerOne to gain an insight into the report validity for both private and public programs. According to ibid., p. 378, the following terms describe the report validity:

- *Noise* references reports which describe vulnerabilities with no security risk.
- *Informative* references reports which describe a security vulnerability which is not regarded as a security vulnerability by the vendor or triage.
- *Duplicate* references duplicate reports of valid vulnerability.
- *Valid* references vulnerabilities regarded as security vulnerabilities by the vendor and triage which are not duplicates.

As Figure 2.3 depicts, the number of reports considered to be *noise* in public reports is comparably high. The percentage of *noise* reports steadily decreased from approximately 40% to 27% from January 2015 to April 2016. However, the number of reports considered *informative* increased from approximately 20% to 40% from January 2015 to April 2016. It can therefore be concluded that reports are more likely to be considered to be *informative* than *noise*. The number of reports considered as *duplicate* dropped from approximately 16% to 10% between January 2015 and April 2016, whereas the number of *valid* reports varied between approximately 24% and 15%.
Compared to the statistics for public reports (Figure 2.3), the reports in private programs suffer from less noise and have a higher rate of valid reports (see Figure 2.4). The *noise* rate for reports of private programs is steadily at approximately 10%, which is between 30% and 17% lower compared to public programs. The number of reports referenced as *informative* was at its lowest in January 2015 with approximately 15%

of reports. However, this changed between February 2015 and April 2015, when 35% of reports were considered to *informative*. It can therefore be concluded that private programs, as well as public programs have roughly the same percentage of *informative* reports. The rate of *duplicate* reports varied between 25% and 10%. However, the average of *duplicate* reports have been approximately 10%, which is almost the same as in public programs. However, the rate of *valid* reports is much higher in private programs than public programs. The rate of *valid* reports varied between 60% and 30% with an average rate being approximately 45%. Compared to public programs, the rate of valid reports is about 16% higher in private programs.
Based on this data, researchers are more likely to achieve a reward in private programs than in public programs. Vendors and triage must handle fewer reports submitted which are considered to be duplicate or invalid. In private programs, the vendor establishes requirements for researchers, and not every researcher is able to conduct research in these programs. Since the researchers are selected prior to joining the program, the number of individuals with a lower skill set is lower than on public programs. The report quality in private programs is therefore higher than in public programs.

Platform	Duplicate	Valid
HackerOne	13%	25%
BugCrowd	36.23%	18.5%

Table 2.4: Percentage of reports on HackerOne and BugCrowd considered as valid or duplicate according to Zhao, Laszka & Grossklags, 2017, p. 376

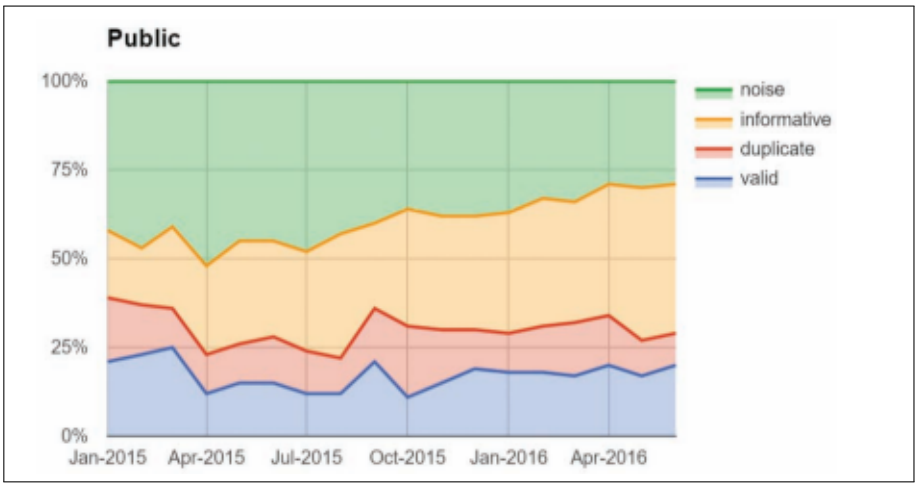

Figure 2.3: Statistics regarding report validity on public programs according to Zhao, Laszka & Grossklags, 2017, p. 378.

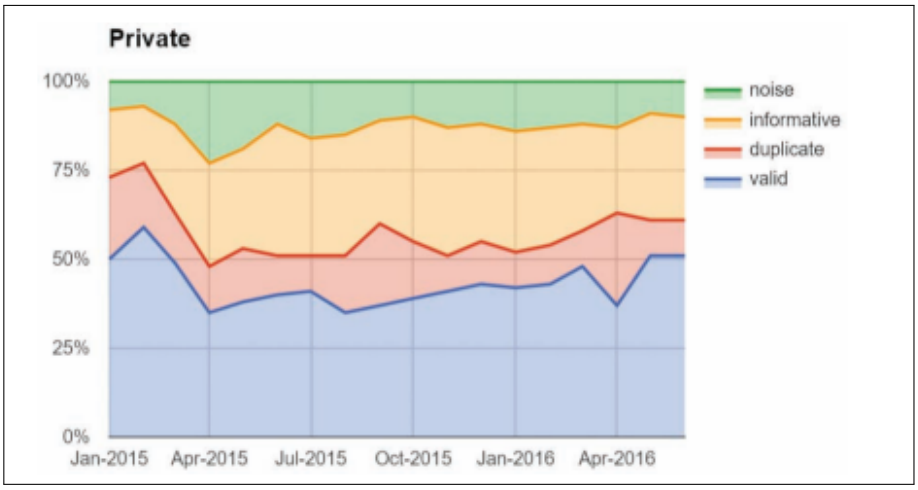

Figure 2.4: Statistics regarding report validity on private programs according to Zhao, Laszka & Grossklags, 2017, p. 378.

2.2 Automated approaches

Since bug bounty programs often offer high monetary rewards, researches are keen to create an automated toolchain for repetitive tasks. The toolchain mainly consists of tools which are executed consecutively while preserving the information that each tool delivered. The toolchain creation ranges from the text-based approach of aggregating tools using bash to full-blown scanners implementing a user interface and backend database. According to Malladi & Subramanian, 2020, p. 38, the development of automated scripts in the context of bug bounty hunting is a best practice for certain scenarios.
As m8r0wn, 2021 describes such automation mostly focuses on the following points:

- Easily identified vulnerabilities
- Continuous reconnaissance
- Maximization of time and profit for repetitive tasks

A common approach is the automation of bug bounty hunting by implementing a simple toolchain using bash. (Hakluke, 2021 andm8r0wn, 2021) However, as Hakluke, 2021 mentions, such scripts are prone to errors and are too slow in the context of larger bug bounty programs. Moreover, such automation stores data in text files, which causes a great deal of read and write operations on the filesystem and leads to the findings being difficult to identify, as well as increasing the systems load which therefore becomes comparably slow.
To manage this problem, ibid. proposed another solution: a Django[5]-based framework

[5]https://www.djangoproject.com/

to discover vulnerabilities. According to Hakluke, 2021, Django was chosen as the framework since most of the used tools were written in Python and Django is a Python based web framework which integrates external Python tools easily. Figure 2.5 de-

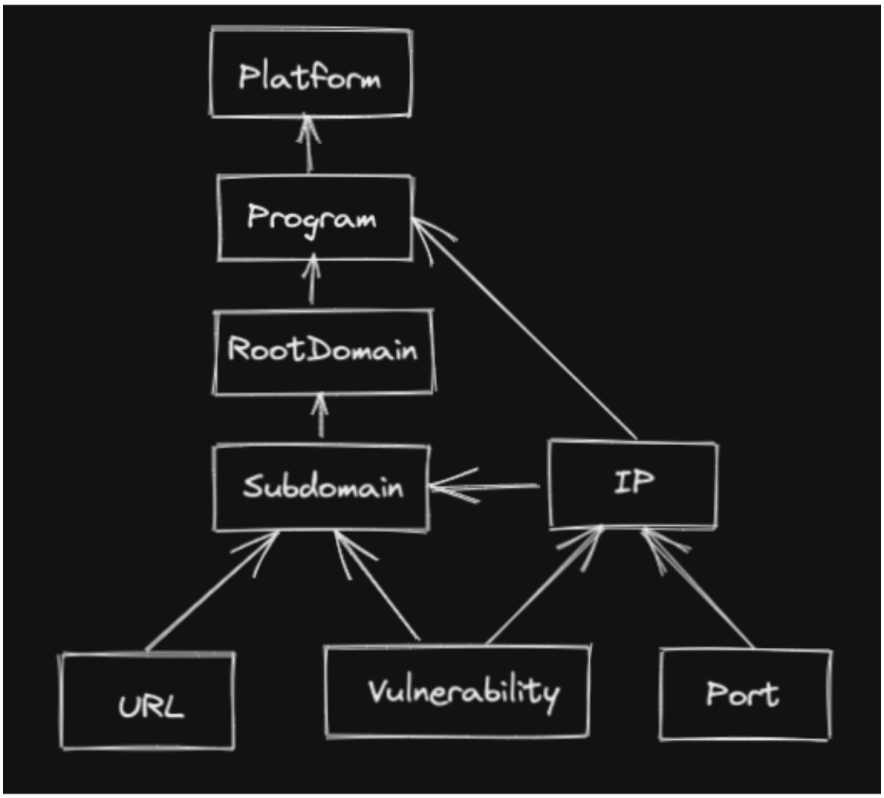

Figure 2.5: The data structure of an automated framework using Django according to Hakluke, 2021.

picts the proposed data structure for the implementation of a Django-based bug bounty hunting automation. *Subdomains* are mapped to their corresponding *root domains* and *IP addresses*. *Vulnerabilities*, *Uniform Resource Locators (URLs)*, and *ports* are also mapped to the subdomains. Vulnerabilities and ports can therefore easily be queried from the database. *Program* references the name of the program on the bug bounty hunting platform.
ibid. developed modules based on this infrastructure which would execute various tools. The output of these tools would be aggregated to the database. According to ibid., this type of automation worked better than the original bash-based solution. However, it was still comparably slow. Originally, this prototype worked in a single-threaded manner. Therefore, scanning two million subdomains with an approximated scanning time of one minute per subdomain would take about four years. (ibid.) The architecture of this prototype was thus changed to a multithreaded approach. With multithreading in place, this solution became increasingly fast. However, the approach

of multithreading typically leads to another problem: resource consumption. Hakluke, 2021 observed that with multithreading in place, the resources of the server running the prototype were exhausted. This approach therefore does not seem feasible since it does not scale to larger bug bounty programs.
To solve this dilemma, ibid. proposed the integration of RabbitMQ[6]. Figure 2.6 depicts

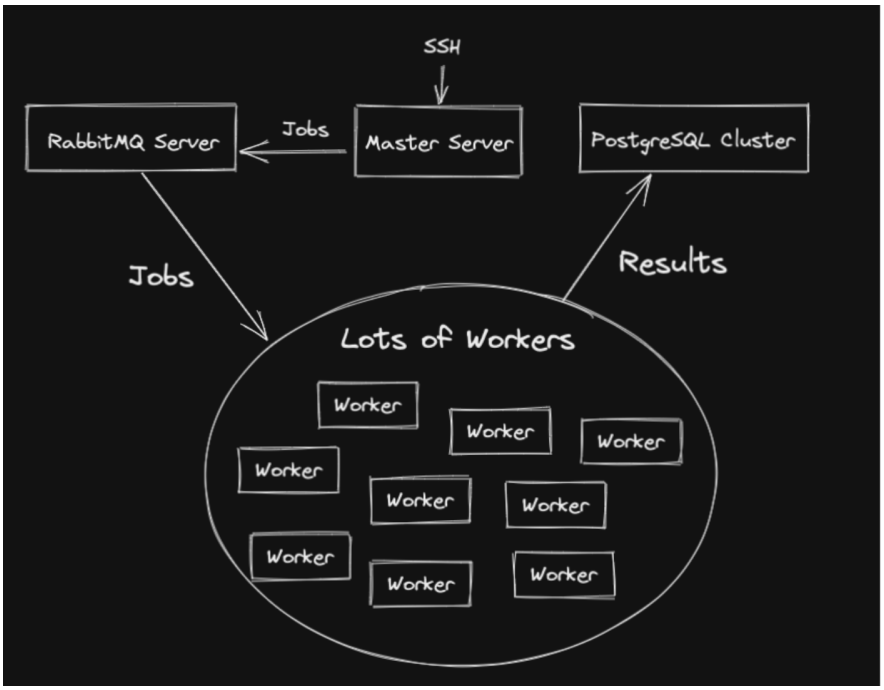

Figure 2.6: Architecture of an automated bug bounty hunting approach using RabbitMQ according to Hakluke, 2021.

the proposed architecture of an automated bug bounty hunting approach using the concept of message queuing. The proposed architecture consists of a *master server*, a *RabbitMQ server*, a large amount of *workers* and a *PostgreSQL cluster*. The *master server* is responsible for defining jobs in the context of bug bounty hunting. Each job consists of a task which must be fulfilled by the workers. The *master server* propagates the defined jobs to the *RabbitMQ server*, which serves as a broker for all job types. It is noteworthy that ibid. does not describe how jobs were distributed and which kind of message exchange was implemented. Jobs were distributed by the *RabbitMQ server* and executed on *workers*. The implementation can therefore be scaled by adding additional *workers* without the need for code changes. (ibid.) *Workers* propagate the results of their jobs to a *PostgreSQL cluster* which stores all the results. The results of the execution can therefore be queried from the *PostgreSQL cluster* without paying attention to the *workers*. Although this approaches works faster and addresses the problems introduced by multithreading, the codebase was still monolithic, certain tools did not

[6]https://www.rabbitmq.com/

integrate well with other tools without developing custom modules and workers proved to be hard to monitor. (Hakluke, 2021)
Another tactic that ibid. introduces is to implement modules for certain tools which can be used in independently or chained together as part of an automation for bug bounty hunting. According to ibid., such approach has to consist of the following systems:

- A data storage system
- A system to manage scaling
- A system to scan for vulnerabilities
- A system to send notifications when vulnerabilities are discovered

According to ibid. Nuclei[7] addresses the last two requirements (see 3.6.1.12).
Another interesting approach introduced by Kiwi, 2021 focuses on authenticated scanning rather than unauthenticated scanning. The key features of the process described by ibid. are the following:

- Only aiming for high-quality bugs. Only the following vulnerabilities are considered: Remote Code Execution (RCE), SQLi, SSRF and XSS.
- Fully automated: the implementation should require a minimum input required from the user.
- Only authenticated scanning
- Reproduceable infrastructure. The components of the infrastructure should be easy to set up and put down.
- Not sophisticated: employs simple techniques proven to find vulnerabilities.
- Distributed: workloads should be distributed across worker nodes.

Figure 2.7 depicts the necessary infrastructure for an automated bug bounty hunting approach as described by ibid. According to this proposed infrastructure, the *http proxy server* is required to distribute the workload among the *worker* processes. *Workers* are in charge of sending requests to the destination and returning the data. *Workers* make use of *Crawlers* and *fuzzers*. *Crawlers* are required to perform the authentication and performing crawling. *Fuzzers* perform injection based attacks as well as pingback-based attacks. The *database* is required to store credentials as well as vulnerabilities. A *login manager* needs to be in place in order to reliably authenticate to the target systems.

[7] https://github.com/projectdiscovery/nuclei

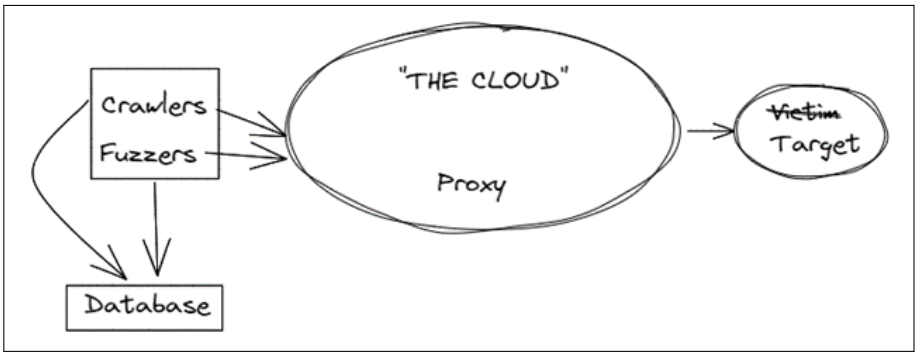

Figure 2.7: Infrastructure necessary for an automated bug bounty hunting approach as described by Kiwi, 2021.

The implementation of the prototype in this thesis focuses on the last approach introduced by Hakluke, 2021 as well the approach discussed by Kiwi, 2021. (see chapter 3)

2.3 OWASP Top 10

The OWASP is a non-profit foundation which focuses on the improvement on software security. (OWASP Foundation, 2022l) With the help of a community consisting of software development professionals OWASP plans to improve the overall security of web applications. OWASP has developed many guidelines to improve software security. The most well-known guideline are the so-called OWASP Top 10 which separate the most common security risks for web applications into categories. The categories are rated based on community ratings. (OWASP Foundation, 2022k) Categories are rated based on community votes since field tests are complicated to realise for a representative amount of web applications. (ibid.) Security researchers are therefore questioned and vote for the most common security risks they identified during studies or practical penetration tests.
The categories of the OWASP Top 10 group Common Weakness Enumeration (CWE) into a common class. Previously, approximately 30 CWEs were grouped into different categories. However, as of 2021, approximately 400 different CWEs are grouped into common categories. (ibid.) In general, CWEs fall into two categories: *root causes* and *symptoms*. *Root causes* are the actual cause of many *symptoms*. *Symtpoms* may have different *root causes* based on their concrete context. The CWEs grouped in the OWASP Top 10 are classified as *root causes* since the *symptoms* may vary based on the context. (ibid.)
Categories are rated by their *exploit* likelihood as well as their *technical impact*. Vulnerabilities are generally rated using the CVSS scoring format. However, there are

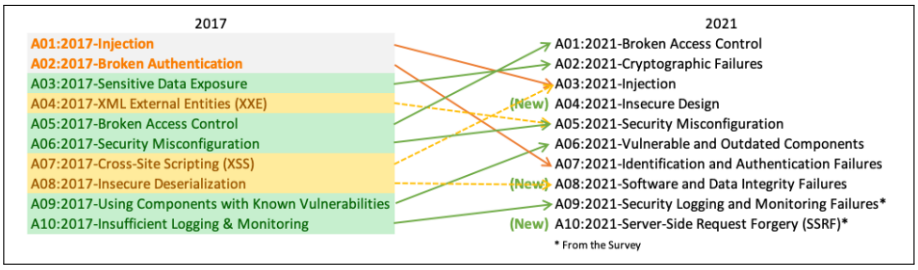

Figure 2.8: Comparison of the OWASP Top 10 between 2017 and 2021 according to OWASP Foundation, 2022k

different versions of the CVSS format. Using CVSS2 *exploit* as well as *impact* have a range from 0.0 to 10.0. The formula used in the CVSS2 calculation, however, weights 60% for *exploit* and 40% for *impact*. (FiRST, 2022a) In CVSS3 *exploit* could score between 0.0 and 6.0, whereas *impact* can be scored between 0.0 and 4.0. (FiRST, 2022b) The weighting remained the same as in CVSS2. The CVSS2 and CVSS3 scores therefore cannot be simply compared since the changes in CVSS3+ caused a shift in the actual calculation.

To effectively rate categories, a formula must be utilised. To determine the rating of a category, the data submitted by professionals in a survey are rated in a specific way. All Common Vulnerabilities and Exposures (CVE)s with their corresponding CVSS scores are grouped by CWEs the weighted *exploit* and *impact* score of the percentage with available CVSS3+ scores are subsequently weighted alongside the vulnerabilities which only have a CVSS2 score available. (OWASP Foundation, 2022k) With these datasets, it is possible to determine an average score. These average scores are mapped to CWEs in the dataset to use them as *exploit* and *technical impact* scoring. (ibid.)

Figure 2.8 depicts the changes from the previous OWASP Top 10 published in 2017 with the current OWASP Top 10 published in 2021.

The following sections discuss the categories listed in Figure 2.8 in depth. However, no attention is paid to the mitigation techniques of the categories since this thesis focuses on the offensive part rather than the defensive part. All categories are listed alongside their metrics relevant for the ranking, as well as the typical causes of the vulnerabilities in the categories as described by OWASP.

2.3.1 A01:2021 – Broken Access Control

As of 2021, the OWASP Top 10 Category Broken Access control moved from fifth to first place. This category has a maximal coverage rate of 94.55%, which means that over 94% of web applications were tested with an average of 3.81% of applications being prone to a broken access control mechanism. (see Table 2.5) It is also noteworthy

how many total occurrences and CVEs are mapped to this particular category. Average ratings for both exploit and impact are considerably high.

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Avg Weighted Exploit	Avg Weighted Impact	Max Coverage	Avg Coverage	Total Occurences	Total CVEs
34	55.97%	3.81%	6.92	5.93	94.55%	47.72%	318487	19013

Table 2.5: Influencing factors for the OWASP Category Broken Access Control as described by OWASP Foundation, 2022a.

Access policies are in place to enforce policies which restrict users from accessing certain data or functions. CWEs mapped in this category reach from information disclosure towards the destruction of data outside the users capabilities.
According to OWASP Foundation, 2022a access control vulnerabilities typically include the following:

- Violation of principle of the least privilege or deny by default
- Bypassing access control checks
- Insecure direct object references
- Missing access controls on Application Programming Interface (API) functions which communicate using POST, PUT and DELETE Hypertext Transfer Protocol (HTTP) verbs
- Privilege escalation
- Metadata manipulation
- CORS misconfigurations which allow API access from untrusted origins
- Forced browsing to authenticated pages without sufficient privileges

2.3.2 A02:2021 – Cryptographic Failures

As of 2021 Cryptographic Failures moved from third to second place. This category was previously known as Sensitive Data Exposure. However, since sensitive data exposure is a broad symptom rather than a root cause, the focus has shifted towards weak cryptographic procedures, which often lead to the exposure of sensitive data. (OWASP Foundation, 2022b) Table 2.6 lists the relevant data for the ranking of this category. Compared to the top category Broken Access Control fewer CVEs are mapped to this

category. However, the average scores for both exploit and impact are higher compared to the leading category.

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Avg Weighted Exploit	Avg Weighted Impact	Max Coverage	Avg Coverage	Total Occurences	Total CVEs
29	46.44%	4.49%	7.29	6.81	79.33%	34.85%	233788	3075

Table 2.6: Influencing factors for the OWASP Category Cryptographic Failures as described by OWASP Foundation, 2022b.

This category covers both data-in-transit and data-at-rest encryption. Therefore, data stored in a database are considered, as well as data transmitted through protocols such as HTTP. Special regard is paid towards highly confidential data such as passwords, credit card numbers, and health records. According to OWASP Foundation, 2022b this category covers aspects such as:

- Data transmission in clear text
- Weak cryptographic procedures and protocols
- Usage of cryptographic keys
- Enforced encryption
- Validated trust chain
- Usage, reuse, and cryptographic sufficiency of initialisation vectors
- Usage of passwords as cryptographic keys
- Sufficient randomness
- Usage of hash functions
- Cryptographic padding methods
- Exploitable side channel information

2.3.3 A03:2021 – Injection

In contrast to 2017, the Injection category moved from first to the third place. According to Table 2.7, 94% of the total web applications were tested for injection-based vulnerabilities, with an average incidence of 3.37%. Compared to the categories placed

higher in this ranking, the average weighted value for exploit is higher than for the Broken Access Control category and the average weighted impact rating is higher than any higher-rated categories. This category covers types of injection based attacks such as XSS and SQLi. (OWASP Foundation, 2022c)

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Avg Weighted Exploit	Avg Weighted Impact	Max Coverage	Avg Coverage	Total Occurences	Total CVEs
33	19.09%	3.37%	7.25	7.25	94.04%	47.90%	274228	32078

Table 2.7: Influencing factors for the OWASP Category Injections as described by OWASP Foundation, 2022c.

According to ibid., applications are prone to injection attacks if the fulfil one of the following requirements:

- Lack of validation, filtering or sanitisation of user supplied data
- Calling non parameterised queries without adequate sanitisation
- Usage of hostile data within Object Relational Mapping (ORM) search patterns
- Hostile data is concatenated with Structured Query Language (SQL) statements without adequate sanitisation

2.3.4 A04:2021 – Insecure Design

This category was introduced in the OWASP Top 10 of 2021. In contrast to other categories, it focuses on design and architecture problems rather than general vulnerabilities. This category maps CWEs and CVEs which are symptoms caused by problems in the design principle of applications (see Table 2.8). OWASP Foundation, 2022d note that thread modelling and secure design patterns should be integrated into software to eliminate risks related to this category during the design phase.

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Avg Weighted Exploit	Avg Weighted Impact	Max Coverage	Avg Coverage	Total Occurences	Total CVEs
40	24.19%	3.00%	6.46	6.78	77.25%	42.51%	262407	2691

Table 2.8: Influencing factors for the OWASP Category Insecure Design as described by OWASP Foundation, 2022d.

As mentioned previously this category accumulates all symptoms caused by underlying design problems. OWASP Foundation, 2022d identifies the difference between insecure implementation and insecure design. The latter may cause insecure implementation. However, an insecure design cannot be mitigated by a perfect implementation since security concerns are not adequately addressed in the first place. OWASP cannot offer any in-depth descriptions of insecure design since this is heavily based on the concrete application as well as its infrastructure. Only a general description can be given for such vulnerabilities.

2.3.5 A05:2021 – Security Misconfiguration

Compared to the statistics of 2017 Security Misconfigurations moved from sixth to the fifth place. Nearly 90% of web applications were tested for such misconfigurations with an average incidence rate of 4.51% (see Table 2.9). The average weighted exploit for this category is quite high, reaching a value of 8.12. Twenty CWEs and 789 CVEs are mapped to this category.

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Avg Weighted Exploit	Avg Weighted Impact	Max Coverage	Avg Coverage	Total Occurences	Total CVEs
20	19.84%	4.51%	8.12	6.56	89.58%	44.84%	208387	789

Table 2.9: Influencing factors for the OWASP Category Security Misconfigurations as described by OWASP Foundation, 2022e.

According to OWASP Foundation, 2022e applications may be vulnerable if the fulfil one of the following criteria:

- Missing security hardening in any part of the application or improperly configured permissions for cloud services
- Unnecessary features are enabled or installed
- Default accounts with default passwords are enabled
- Revelation of stack traces caused by insufficient exception handling
- Security features of systems are deactivated or not configured securely
- Security settings of software and infrastructure are not configured sufficiently
- Lack of security headers

- Usage of outdated software

This category includes the usage of outdated software a category is dedicated to outdated components (see 2.3.6), because the usage to outdated software also proposes a security risk for the applications and is a security misconfiguration.

2.3.6 A06:2021 – Vulnerable and Outdated Components

This category was previously known as "Using Components with Known Vulnerabilities". Compared with the statistics of 2017 this category moved from ninth to sixth place. (OWASP Foundation, 2022f) Only three CWEs mapped to this category (see Table 2.10). However, no dedicated CVEs are mapped to this category, the average weighted exploit and impact thus cannot be rated right away. According to ibid. average weighted exploit and impact are rated with a score of 5.0 per default. Moreover, 51.78% of applications were tested for outdated and vulnerable components with an average incidence of 8.77%.

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Avg Weighted Exploit	Avg Weighted Impact	Max Coverage	Avg Coverage	Total Occurences	Total CVEs
3	27.96%	8.77%	5.0	5.0	51.78%	22.47%	30457	0

Table 2.10: Influencing factors for the OWASP Vulnerable and Outdated Components category as described by OWASP Foundation, 2022f.

According to ibid. applications are likely to be vulnerable if one of the following criteria is fulfilled:

- The version of the utilised components are unknown
- The software is vulnerable, unsupported, or outdated. This also includes the infrastructure, therefore, the operating system of the webserver, Database Management System (DBMS), applications, APIs, runtime, and libraries are considered as well.
- Lack of vulnerability scans for used components
- Lack of patching
- Missing compatibility tests
- Unsecure component configuration

The cause of unsecure component configuration is also related to the Security Misconfiguration category (see 2.3.5).

2.3.7 A07:2021 – Identification and Authentication Failures

This category was previously known as Broken Authentication. Compared to 2017 this category moved from second to seventh pace. (OWASP Foundation, 2022g) CWEs related to identification failures are mapped to this category (e.g. identity, authentication, and session management). Table 2.11 describes the indicators used for the rating of this category. 79.51% of applications were tested for vulnerabilities in this category with an average incidence of 2.55%. The average weighted exploit is comparably high with a value of 7.40, however, the average weighted impact only reaches a value of 6.50.

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Avg Weighted Exploit	Avg Weighted Impact	Max Coverage	Avg Coverage	Total Occurences	Total CVEs
22	14.84%	2.55%	7.40	6.50	79.51%	45.72%	137195	3897

Table 2.11: Influencing factors for the OWASP Category Identification and Authentication Failures as described by OWASP Foundation, 2022g.

According to ibid. application may be prone to authentication weaknesses if one of the following criteria is fulfilled.

- Attacks such as credential stuffing are not mitigated
- Brute force attacks are not mitigated
- Weak password policy or use of default credentials
- Usage of weak or ineffective credential recovery mechanisms
- Passwords stored as plaintext or weakly hashed
- Missing or ineffective two-factor authentication
- Usage of session identifiers in query parameters of HTTP GET requests
- Reuse of the session identifier
- Incorrect invalidation of session identifiers

This category also cross-references another category. The risk factor "Storage of passwords in plain text or weakly hashed" is directly related to the category Cryptographic Failures (see 2.3.2).

2.3.8 A08:2021 – Software and Data Integrity Failures

This category aims to address the process of software updates, critical data, and Continuous Integration / Continuous Delivery (CI/CD) pipelines without integrity verification. (OWASP Foundation, 2022h) This category has the highest average weighted impact rating of all categories within this statistic (see Table 2.12). 75.05% of applications were tested for this type of vulnerabilities with an average incidence rate of 2.05%. This category also maps 1152 CVEs as a unique category. This category also serves as a kind of aggregation of various categories. For example, this category includes everything related to serialisation. (ibid.)

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Avg Weighted Exploit	Avg Weighted Impact	Max Coverage	Avg Coverage	Total Occurences	Total CVEs
10	16.67%	2.05%	6.94	7.94	75.04%	45.35%	47972	1152

Table 2.12: Influencing factors for the OWASP Software and Data Integrity Failures category as described by OWASP Foundation, 2022h.

One of the major focuses of this category are data integrity failures due to the software and infrastructure not verifying the integrity of external data. (ibid.) An example would be an application relying on third-party plugins which are imported from untrusted sources which may therefore be compromised. The compromised plugin can have devastating impacts on the overall security of the application which uses it. Another focus of this category are issues related to CI/CD. Improperly configured CI/CD pipelines may introduce several vulnerabilities such as unauthorised access to the source code or the injection of malicious code. Software often includes automatic updates, however, these updates may not be checked for their integrity, which allows attackers to spoof malicious updates. Since many applications are dependent on serialisation, serialisation is also an attack vector covered in this category. Insecure serialisation may allow the attackers to inject their own code in the context of the application and execute it.
Since all topics covered in this category are dependent on the actual infrastructure, ibid. does not give concrete descriptions of injection points or attack scenarios. Descriptions of vulnerabilities are general.

2.3.9 A09:2021 – Security Logging and Monitoring Failures

This category has been renamed from "Insufficient Logging and Monitoring" and covers all the vulnerabilities which are caused by either insecure logging or failures of

logging. (OWASP Foundation, 2022k) Only four CWEs are mapped to this category (see Table 2.13). However, the average weighted exploit is high for the placement of this category. Only 53.67% of applications were tested for this category, reaching an average incidence rating of 6.51%.

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Avg Weighted Exploit	Avg Weighted Impact	Max Coverage	Avg Coverage	Total Occurences	Total CVEs
4	19.23%	6.51%	6.87	4.99	53.67%	39.97%	53615	242

Table 2.13: Influencing factors for the OWASP category Security Logging and Monitoring Failures as described by OWASP Foundation, 2022i.

As OWASP Foundation, 2022i notes, vulnerabilities in this category are often difficult to verify. Verification of such vulnerabilities often requires interviews regarding the state of the employed logging and the detection of attacks during a penetration test. This category is focused on detecting, escalating and responding to active breaches which cannot be done without the employment of sufficient logging. (ibid.)
ibid. notes that applications fulfilling one of the following criteria are likely to be prone to a vulnerability in this category:

- Not logging logins, failed logins, and high value transactions
- Missing or inadequate logging of errors and warnings
- Lacking monitoring of API log files
- Storing log files solely on local machines
- Inappropriate alerting thresholds and lacking escalation processes
- Failure of the application to detect, escalate or alert for active attacks in real time

However, even sufficient logging may lead to information leakage which leads to vulnerabilities as described in Broken Access Control (see 2.3.1).

2.3.10 A10:2021 – Server-Side Request Forgery (SSRF)

In contrast to the other categories in this list this category proves to be quite special. This category is solely based on the community survey which was keen on adding issues regarding SSRF to the OWASP Top 10. (OWASP Foundation, 2022k) Table 2.14 depicts the metrics associated with this category. It shows that both the average weighted exploit rating and average weighted impact rating are well above

average. Additionally, 67.72% of applications were tested for vulnerabilities in this category with an average incidence rate of 2.72%. This category only consists of a single CWE which maps to 385 CVEs.

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Avg Weighted Exploit	Avg Weighted Impact	Max Coverage	Avg Coverage	Total Occurences	Total CVEs
1	2.72%	2.72%	8.28	6.72	67.72%	67.72%	9503	385

Table 2.14: Influencing factors for the OWASP Category Server-Side Request Forgery (SSRF) as described by OWASP Foundation, 2022j.

SSRF vulnerabilities describe the procedure of a web application fetching data from a user supplied Uniform Resource Locator (URL) on the server side. Since the server rather than the client executes the request, an attacker gains access to the internal network of the server and may be able to bypass the protection of firewalls since the request is from the internal network. According to OWASP Foundation, 2022j fetching user supplied URLs is becoming an increasingly common scenario along with architectures becoming more complex and services migrating to the cloud. Therefore, SSRF incidence rates as well as severity ratings are increasing.

2.4 Internet of Things in Bug Bounty Hunting

In recent years the topic of IoT has attracted attention due to the growing number of security vulnerabilities in IoT devices. (Ding, Jesus & Janssen, 2019, p. 49) Such devices consist of hardware and software which may both be prone to security vulnerabilities. The rapid growth of interest in IoT vulnerabilities in such devices typically leads to a large audience being affected by them.

IoT devices are used by many people to automate aspects of their daily life. Examples of such automation range from a smart toaster to smart industries. With IoT devices being used in critical areas such as smart industries, vulnerabilities affect the devices' security, safety and reliability. (ibid., p. 49) With the wide variety of use cases for IoT devices, their complexity has also increased. Increased complexity typically leads to software and hardware being prone to security vulnerabilities since not all abuse cases are properly addressed or implemented. (ibid., p. 49) IoT devices are reliant on input from both the digital world (e.g. apps) and from the physical world (e.g. heat). Devices must react on the input and automate decisions based on them. For example an internet-connected heat sensor may automatically shut down an industrial machine

if it discovers that the machine overheats. If the heat sensor can be influenced by an attacker, the attacker may spoof values of the heat sensor to either prevent it from shutting down the machine or shutting down the machine prematurely. Such behaviour affects both the business and the staff operating the machine. If the machine is shut down prematurely, the business suffers from a financial impact since the machine has unplanned downtime and production may not be able to continue while the machine is shut down. If the machine overheats the safety impact of the machine is impacted since the staff in the area of the machine is put at risk by the machine catching fire or exploding. The business also suffers from a financial impact in this case since the machine may be destroyed by overheating and production may not be able to continue. According to Ding, Jesus & Janssen, 2019, p. 50 the six most common reasons for higher risks of exposure in the context of IoT are as follows:

- IoT systems do not have well-defined perimeters.
- IoT systems are highly heterogeneous with respect to communication medium and protocol.
- IoT systems often include devices not designed to be secure or connected to the internet of things.
- IoT devices control other devices without human supervision.
- IoT devices are often physically unprotected.
- The large number of connected devices increases the security complexity.

The vulnerability management process of normal IT systems should therefore also be applied to IoT devices. The vulnerability management process consists of five stages:

- Checking for vulnerabilities
- Identifying vulnerabilities
- Verifying vulnerabilities
- Mitigating vulnerabilities
- Patching vulnerabilities

The vulnerability management process is difficult to enforce on IoT devices. The underlying operating systems and firmware of such devices are hard to update. (Zou, 2022) Additionally, these devices are not designed to run third-party software such as antivirus solutions. (ibid.) Especially medical devices are challenging to update as they are regulated by the Food and Drug Administration in the U.S.A who require stringent

procedures for any change. (Zou, 2022) Often vendors also do not employ an automated update mechanism for their IoT devices, which leads to the devices quickly becoming outdated. In contrast to other systems IoT devices and specific medical devices have a longer lifespan of over 10 years of continuous operation. (ibid.) Downtimes of IoT devices controlling other IoT devices caused by patching can prove to be business critical.

According to Zhao, Laszka & Grossklags, 2017, p. 53 there is insufficient literature on how to efficiently employ IoT in the context of bug bounty hunting and ethical hacking. One reason for this is that IoT devices are highly dependent on specific hardware, where vulnerabilities in the hardware are considered the most critical spot. (ibid., p. 53) Once hardware is delivered to the client, the patching of vulnerabilities in hardware can prove challenging. Hardware patching for IoT devices is often not possible in a business-efficient manner. (ibid., p. 53) Another influencing factor is the low regard for security by IoT manufacturers and consumers. (ibid., p. 53) Security aspects like bug bounty hunting are therefore often not considered in the first place. IoT devices are often produced in a manner which proves to be the most business-efficient. Inexpensive components which are not fully tested are often built into IoT devices, which leads to an increased attack vector. (ibid., p. 53)

However, IoT-based bug bounty programs are available on HackerOne. Based on these bug bounty programs, the security of IoT devices is mostly considered by large vendors. For example, Xiaomi offers rewards for vulnerabilities reaching from $150 to $4,000. (One, 2021) However, for most bounty programs for IoT devices, the researcher must purchase a device to examine it which requires a financial commitment by the researcher prior to being able to examine the device for vulnerabilities. Many researchers do not take on such financial commitment, which means that programs for IoT devices are less frequently researched than other programs. On the one hand, this leads to more vulnerabilities being undiscovered in IoT devices. On the other hand, it could prove to be a chance for researchers since they are more likely to discover a vulnerability compared to other programs. From a business perspective no efficient way exists to provide a researcher with a device since this may lead to researchers claiming devices for research purposes without conducting research. This leads to a financial impact on the company. A rental model could prove to be the solution when enforced by the bug bounty intermediary. Using this approach, the researcher is able to test the device for vulnerabilities without a financial commitment for a limited amount of time, and the vendor does not have to provide the device for free. Cases of hardware being destroyed during testing would need to be considered by the bug bounty intermediary and addressed in the rules of engagement.

Chapter 3

Numb4d - Automated bug bounty hunting

3.1 Introduction to the Idea behind this Prototype

With the increased interest in bug bounty hunting worldwide, the desire for an automated approach in this field also increased. Many solutions for the automation of the bug bounty hunting process are presently available, yet many of these approaches are not publicly available. The main reason is that in many bug bounty hunting programs, only the fastest individual to report a certain vulnerability receives the bounty or acknowledgement. If individuals share their automation, they are thus less likely to receive pay-outs since the number of competitors who have the same tooling as the initial developer also increases. Many publicly available approaches therefore cannot compete in the competitive setting of bug bounty hunting.

Some available bug bounty hunting automations are tightly coupled to a specific operating system. For example, BugBuntu[1] may be used to automate the bug bounty hunting process, but it has its own operating system. The researcher who uses such automation is thus bound to a certain operating system to perform tests. This prototype aims to be as flexible as possible. It utilises a client-server technology with a uniform API which allows the researcher to deploy it in each and every Linux environment.

The process of bug bounty hunting is far from static and difficult to automate. Every system under test has different security mechanisms implemented. An automated approach may thus achieve satisfying results in one program but fail to discover any vulnerabilities in another. Therefore, no one-fits-all solution exists in the agile context

[1] https://github.com/halencarjunior/BugBuntu

of bug bounty hunting.
However, there are static components in bug bounty hunting. Since the responses of systems are prone to differ across programs, the main workflow remains the same. This includes the following steps:

- Reconnaissance
- Content discovery
- Testing various attack vectors against the discovered content
- Discovering vulnerabilities based on the tests
- Reporting vulnerabilities

It is possible to automate many techniques in these steps. For example, the process of reconnaissance is nearly the same for every program. The testing of attack vectors utilises tools which may discover vulnerabilities. Such steps can be automated in a way that the researcher is no longer required to start each tool on their own or to analyse every output.
This prototype aims to automate as many steps as possible, including reconnaissance, content discovery, testing attack vectors (i.e. fuzzing) and deriving possible vulnerabilities based on the results of fuzzing. The reporting of vulnerabilities is the key factor in bug bounty hunting. The acceptance of vulnerabilities is heavily influenced by report quality. If the submitted report is of poor quality, the report is less likely to be accepted even if the vulnerability is valid. Due to this problem, the reporting of vulnerabilities is not be automated by this prototype. The researcher must analyse the outcomes which the prototype returns and create a high-quality report based on them. The prototype will report every result to the researcher. Such result consists of the vulnerable system, the attack vector, a proof of concept and the CVE identifier. These results can help the researcher to create a valid report with backing materials and in the best case, a PoC for the identified vulnerability.

3.2 Basic Prototype Requirements

3.2.1 Infrastructure

This prototype consists of three main parts: a client, an application server, and a database server. The client and application server interact using Representational State Transfer (REST). The application server interacts with the database server to cache relevant data.

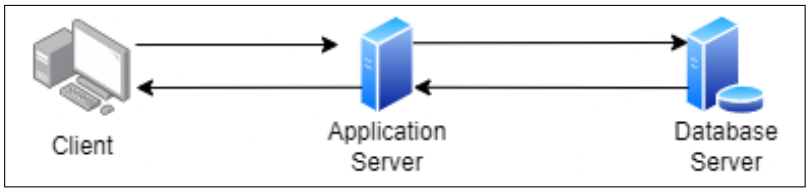

Figure 3.1: The bare minimum requirements for the infrastructure of this prototype

Figure 3.1 depicts the bare minimum requirements for the infrastructure. The client does not have tools installed, all tools are installed on the application server. Any client may thus be used to send requests to the server. The application server offers different APIs for different tools. To test this prototype, the application was supplied with 16 Gigabyte (GB) of Random Access Memory (RAM) and 25 GB of disk storage to allow smooth operations. In contrast to the application server, the database server requires less RAM but more disk storage. During testing the database server was supplied with 4 GB of RAM and 100 GB of disk storage.
The application and database servers are both based on an Ubuntu Server 20.04 image[2]. Neither server requires a graphical user interface. Since the application server is required to install a plethora of tools, an installation script is provided which automates this process. Both servers are run as virtual machines using VMWare Workstation Pro 16.2.1[3]. There are no requirements for the operating system of the database server. However, the application server must be run on a Linux operating system since it uses the Linux path notation to resolve paths on the file system.

3.2.2 Architecture

The prototype consists of a client-server architecture. The application server offers an API for each installed tool on the server. The client is separated from the actual execution of the tools. Since the application server needs a plethora of tools, an installation script is provided which installs each required tool on the server.
The client requests the execution of tools on the server. To achieve this, the client makes use of the REST APIs which the server offers. To invoke such tools, the performs calls using different HTTP methods. The application server offers most of the API methods using the HTTP-POST verb. Selected tools are invoked using the HTTP-GET method. The client holds the information about the system which must be tested and propagates the system to the desired API method. The application server is therefore not required to maintain any state with the client. The interaction between client

[2]https://releases.ubuntu.com/20.04/
[3]https://docs.vmware.com/en/VMware-Workstation-Pro/16.2.1/rn/VMware-Workstation-1621-Pro-Release-Notes.html

and server utilises Javascript Object Notation (JSON) as the serialisation format of the results.
The application server does not know about the actual toolchain. The client application must compose the actual toolchain based on the API methods which the server offers. With this type of separation, the application server is able to be as flexible as possible. For instance, it is possible to only invoke a single tool on the server without executing the whole toolchain, which allows the application server to be independent of the execution context of the bug bounty hunting automation.

3.3 Server Technology

As mentioned in subsection 3.2.2, the application server is responsible for hosting various tools. The application server is responsible for the invocation of these tools, as well. Most tools cannot be simply invoked from a virtual environment since they often require interaction with the operating system (e.g. running multiple threads concurrently within the execution context of a certain tool). These requirements limit the choice of the programming language on the server side. The server side must be implemented so that it is able to perform operating system calls to start tools. While the invoked tool is running, the application must verify the state of execution. It must handle errors in a graceful way while providing the client application with details on why an error occurred.
The programming language on the server side must comply with these requirements, and it should be as lightweight as possible. Therefore, NodeJS was chosen as the JavaScript framework to implement the API to invoke tools on the application server. NodeJS offers rich integration of the operating system and is able to perform calls to the operating system. The code to invoke a tool using native functions of the operating system is listed in 3.1. This listing refers to the invocation of the tool *nmap*.

```
1  const { exec } = require('child_process');
2  let command = 'nmap -sV -pN example.com';
3  exec(command, function (error, stdout, stderror) => ...);
```

Listing 3.1: Example for invoking a tool on the operating system

Aside from the invocation of system commands, the invocation of Python tools is necessary. It is complicated to invoke Python tools which require different Python versions by solely interacting with the operating system using the command line. In addition, many Python tools require a large amount of parameters. Python-shell[4] is a Node Package Manager (npm) package which aims to solve this problem. This package was

[4]https://www.npmjs.com/package/python-shell

also integrated into the prototype to allow smooth handling of Python scripts within the NodeJS runtime. An example for the usage of python-shell is listed in 3.2.

```
1  const pyShell = require("python-shell").PythonShell;
2  let options = {
3        pythonPath: "python3", //The used python environment
4        mode: 'text',
5        scriptPath: executingPath, //path to the script e.g. /etc
           ↪ /script.py
6        args: ['-d', url, '--no-color'] //parameters used for the
           ↪  invocation of the script relates to -d <url> --no
           ↪ -color
7    };
8    pyShell.run(helpers.sublist3rScript, options, function (error
       ↪ , result) //Invocation of the script with callback
       ↪ fucntion
9        {
10           if (error) { //Set if an error occurs during the
                ↪ scripts execution
11             //handle the error
12           }
13           else {
14               //Handle the successful execution
15           }
16       });
```

Listing 3.2: Example for invoking a python tool using python-shell

Another important factor which becomes increasingly relevant during the evaluation is logging. It is required to check which commands were executed and what their actual results were to verify correct parsing of results and correct invocation of the tools. Log files which hold such information must be created on the server. To accomplish that, NodeJS is required to create files on the operating system and write to them. NodeJS offers rich out-of-the-box solutions for such requirements. The built-in fs module may be utilised to create files on the operating system and interact with the created files. Since these files do not reside in the directory of the invoking JavaScript file, the path to the actual log file must be resolved. Helper methods are utilised to convert relative paths to fully qualified paths. The usage of fully qualified paths for writing log files is much more robust than utilising relative paths, which may prove to be a problem once the environment is moved to a different server with a different path structure.

3.4 Database Technology

This prototype interacts with a database which is used to store results found by various tools and to cache results of certain long-running tools to decrease the amount of time required for successive scans in the same bug bounty program. The interaction with the database solely occurs on the server side. The client therefore does not require a connection to the database and does not have to be aware that a database is used to store results. To connect to the database, the NodeJS server requires a connection string to the database stored under `server/credentials/database.txt`. Results retrieved from the database are not marked as cached. Therefore, these results are interpreted like any other result achieved from the execution of a tool. Regarding the technology, PostgreSQL is used as the DBMS. PostgreSQL is an object relational DBMS capable of executing multiple processes at once. PostgreSQL is therefore faster than technologies such as MySQL according to Brandon, 2022. PostgreSQL is the DBMS of choice for applications which require a high amount of read and write transactions. In contrast to other DBMSs PostgreSQL offers a wider variety of datatypes, however, the datatypes used in this prototype are basic datatypes available most other comparable DBMSs. This database management system offers a wide variety of security measurements. In a single database called `masterthesis` only the user *postgre* is able to access this database. The user has both read and write privileges on all tables in this database. However, the user cannot access any other database which may reside on the PostgreSQL server.

3.4.1 Data Model

The data model implemented in this database represents a relational approach to data modelling. Connections between tables are maintained using foreign keys. Therefore, each individual table must reference the primary key of the linked table rather than storing information about the whole system. Figure 3.2 depicts the data model representing the structure of the `masterthesis` database.

Domains is the table referenced by most other tables in the data model. This table stores all information about domains and links the domain to a unique integer-based identifier. Domains are limited by a unique constraint to avoid duplicate domains resulting in ambiguous references.
Wafs is a table responsible for representing the Web Application Firewall (WAF)s detected by WafW00f (see subsubsection 3.6.1.8). A unique integer-based identifier is

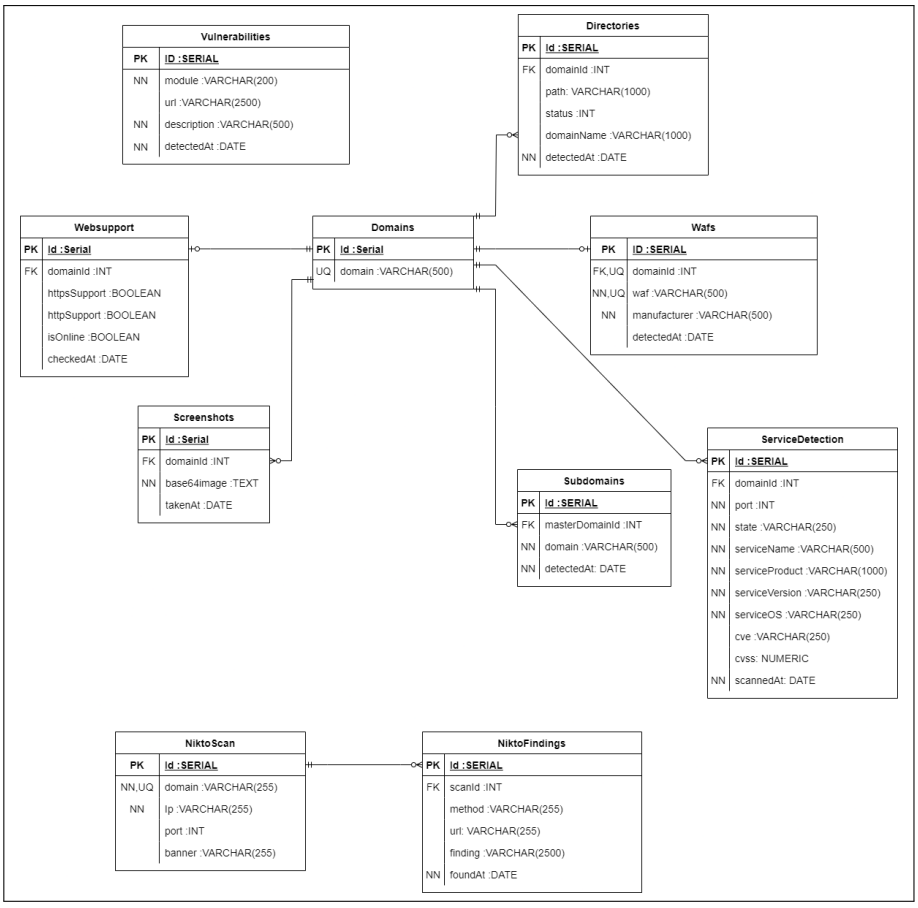

Figure 3.2: Data model used in the database of this prototype

present for every row in this table. Each row is linked to a *domain* by the foreign key constraint *domainId*. The column *waf* represents the actual WAF solution in place. The column *manufacturer* lists the manufacturers of every detected WAF. A *manufacturer* may not be null. The combination of *waf* and *domainId* must be unique since every domain may only have one WAF in place. The column *detectedAt* represents the timestamp when this WAF was detected. *detectedAt* is autogenerated by an INSERT-Trigger. If no WAF was detected by WafW00f, no data will be persisted to the database.

Subdomains represents the links between *domains* and their corresponding subdomains. The column *masterDomainId* represents the link of the subdomain between the subdomain and the actual domain. The *domain* column is somewhat ambiguous, this column represents the name of the actual subdomain rather than the domain to which it is linked. The domain name of a detected subdomain may not be null since only detected subdomain names are written to this table. The *detectedAt* column contains the timestamp when a certain subdomain was detected. The content of this column is automatically generated by a INSERT-Trigger. When subdomains are detected these domains are written to the subdomain table as well as to the domain table itself. This is necessary since most other tables rely on the link to an actual domain rather than a subdomain.

ServiceDetection contains the services detected on a certain domain. Detected services are linked to a domain using *domainId*. The *port*, *state*, *serviceName*, *serviceProduct*, *serviceVersion* and *serviceOS* columns identify the services running on the domain by their characteristics. These columns may not be filled with null values since they are characteristics of a service, however, the *serviceVersion* and *serviceOS* columns are an exception. They may not be filled with null values, but tools like nmap (see subsubsection 3.6.1.13) may not be able to determine such parameters. If those parameters are not detected *serviceVersion* as well as *serviceOS* are substituted with the value −. The *cve* and *cvss* columns can be filled with null values since these attributes are not required to be detected for every service. The *scannedAt* column is automatically generated by an INSERT-Trigger. If no services are detected on a certain domain, no data is persisted to the database.

Screenshots contains the screenshots of a certain domain and is linked to domains by utilising *domainId*. The *base64image* column contains the screenshots of the domains represented in the Base64 encoding. Since the Base64 representation of images may vary in length, the datatype *text* is chosen. This datatype does not limit the maximum length of the content to a certain number of characters. *base64image* representations may not be null since screenshots taken are persisted into the database. The *takenAt*

column is automatically generated by an INSERT-Trigger.

Websupport stores the availability status of certain domains. The web support status is linked to a certain domain by using *domainId*. The *httpsSupport* and *httpSupport* columns are used to determine whether the domain is reachable by the HTTP or Hyptertext Transfer Protocol Secure (HTTPS) protocol. These columns are represented by the boolean datatype, which can take the value of `true` or `false`. The *isOnline* column determines whether the domain was reachable at all. If *isOnline* is set to `false`, the values for *httpsSupport* and *httpSupport* must be `false`. The *checkedAt* column is automatically generated by an INSERT-Trigger.

Directories contains all directories which were detected for a certain domain by utilising tools such as gobuster (see subsubsection 3.6.1.9). Discovered directories are linked to domains by a foreign key constraint on *domainId*. The *path* column stores all the discovered paths on such domain. The *status* column lists the returned HTTP status code for each discovered directory and *detectedAt* is automatically generated by an INSERT-Trigger.

Vulnerabilities is a table used to store vulnerabilities detected by different tools. This table is not linked to the *Domains* table. It solely serves the purpose to generically store vulnerabilities without the requirement to link these vulnerabilities to the actual domains. Vulnerabilities are identified by an unique integer-based identifier. The *module* column is used to identify the tool which reported the vulnerability. A *module* may not be null since all vulnerabilities must be reported by a tool to be valid. The *url* column identifies the location of the actual vulnerability. Since vulnerabilities may be bound to a certain path within the web application, it is necessary to link the required path. The *description* column stores the descriptions of the actual vulnerabilities. This column may not only be used to store descriptions, but also used to store results of a certain *url*. For example, Nuclei (see subsubsection 3.6.1.12) stores the JSON description for a vulnerability in the *description* column. The column *detectedAt* is automatically generated by an INSERT-Trigger.

NiktoScan stores all scans performed by Nikto. This table is only required to store the information of actual vulnerabilities detected by Nikto. It only stores information about the actual target. The *domain* column stores information about the scanned domain. *IP* stores the actual Internet Protocol (IP) address of the scanned domain. This information may be used in further testing. The *port* column represents the port which was scanned. Information about the service running on a certain port is stored in the *banner* column.

The **NiktoFindings** table is utilised to store vulnerabilities discovered by Nikto and is linked to *NiktoScan* table by the use of a foreign key constraint on the *scanId* column.

The *method* column represents the actual method Nikto used to discover the vulnerability. The *url* column stores the URL which is vulnerable to the *method* performed by Nikto. The *finding* column stores all information about the finding discovered by Nikto. The finding is represented in a text-based manner and stores information about the actual finding alongside a description of the actual finding. The *foundAt* column is automatically generated by an INSERT-Trigger.

3.4.2 Caching

As mentioned previously, this prototype implements caching. Data are stored in the database and subsequently fetched and provided to the corresponding client. Caching is employed since it limits the intrusiveness of certain tools. For example, nmap scans are long-running and noisy. By employing caching, such scans only have to be executed a single time, and the results may later be fetched by the server. The target therefore is not scanned a second time and no additional traffic to the target is caused. To employ caching in a useful manner, all tables have a column which stores the date when a certain data row was added to the table. This date can be utilised to decide whether the cached data should be returned to the client or be deleted to return the most recent data. The addition of caching can lead to false positives when certain changes to the scanned infrastructure occur. For example, when the service version running on a certain port of the target system is updated to a version that eliminates the detected CVEs, the data in the cache will still return the vulnerable version since it was not updated, and the cached information including the old service version is passed to the client. Data in the cache should therefore be validated based on the time of the detection. As a rule of thumb, cached data older than one week should not be returned to the client since this data may lead to false assumptions regarding the vulnerable state of the target. Data which have a detection date older than a certain time span should be removed or moved to another database which stores the historic details of targets.
Another important use for caching is the possibility to restore the state of a scan which was aborted prior to finishing. The server is able to fetch data from the database and return the results stored in the database to the client. The client can thus restore a certain state without driving traffic to the target. Using this functionality, reports of a certain target may be restored. However, this functionality is not completely implemented into this prototype. This prototype mainly uses the cached data to avoid scans such as enumerating subdomains or scanning the target for open ports and services. The vulnerabilities stored in the database are not retrieved since this data are solely used for historic purposes, as detected vulnerabilities may change rapidly. It would not make sense to retrieve actual web-application-based vulnerabilities from the cache

since these vulnerabilities are often eliminated by simple configuration changes or software updates which may occur in production without consequences for other parts of the web application.

3.5 Client Technologies

The proposed prototype consists of two main components: a client and a server. The client is responsible for building the toolchain using the uniform API interfaces of the servers' implementation. The server is required to perform the invocation of the actual tools and presenting the tools' output in a way that can be interpreted by the client. Client and server therefore communicate using a JSON-based REST API implementation.
As mentioned previously, the client functions decoupled from the actual tools, and the requirements for the client are rather limited. A client implementation must fulfil the following requirements:

- Capability to invoke a JSON-based API
- Capability to interpret and parse JSON
- Support of parallelisation
- Capability of presenting results in a human-readable form
- Capability of configuring the actual workflow

A client can be implemented in any language which fulfils the afore mentioned requirements. The client of the prototype was implemented in two programming languages. On client is implemented in Python, which allows for better parallelisation and running without an actual user interface. The second client is implemented using Angular. This client consists of a web-based user interface which is capable of displaying findings in real time.

3.5.1 Python Client

As mentioned, one of the clients is implemented in Python. This client can be run without user interaction and does not display results in real time, but rather pursues the principle of executing long-running scans. Results of these scans are later rendered in a human-readable form.
This client requires Python in version 3.x since it relies on functions and syntax which was added with the release of Python 3.x. The client was tested in a Python 3.9 envi-

ronment and showed no issues. Testing for earlier Python 3.x versions was not done, however, the client should not have any issues with lower Python 3.x versions.

3.5.1.1 Software Design

This client is implemented in an object-orientated approach. It therefore utilises classes which are required for the invocation of tools. Four types of objects are distinguished:

- API wrappers
- Workflow components
- Workflow builder
- Main Entry point

API wrappers are determined to implement the logic necessary to communicate with the API. Such classes abstract the actual API call necessary for the invocation of the desired tool from the workflow's logic. These classes are instantiated when a tool is invoked with a specific payload and are required to hold their results in a private variable. Results may only be delivered by getter methods. API wrappers depend on the requests[5] library. The requests library fully implements the HTTP standard and is able to execute requests. Since this library is able to feature full HTTP support, it is also able to fully integrate with JSON. Therefore, the API wrappers do not need to parse an object to JSON to execute a request which requires a JSON body or delivers its result in the form of a JSON response body. The class only needs to prepare a dictionary which holds the required parameters for the request, as the parsing is done by the requests library. When the API call returns a JSON response body, the invoking class can handle the response body like a usual dictionary. The API wrapper must react to the different response statuses returned by the API call as described in section A.1. Three different status codes are distinguished:

- `200 OK`
- `417 Expectation Failed`
- `500 Internal Server Error`

When the status code `417 Expectation Failed` or `500 Internal Server Error` occur, no result is produced. API wrappers only collect results with a status code of `200 OK`.

[5]https://pypi.org/project/requests/

API wrappers are also responsible for defining the rendering process. The rendering process is defined by jinja2[6] templates. Each of these classes has a sophisticated jinja2 template defined which describes how results are rendered. Templates are defined in HTML with the possibility of defining, for example, loops and if-statements inside the template. To style the produced results each the UIkit[7] framework is utilised. This framework features the required JavaScript and Cascading Style Sheet (CSS) files to enhance the styling of the results. jinja2 offers a Python library which is used to produce HTML files which display the actual result of API calls. One result file is rendered for each instantiation of the API wrapper.
The client implements a single API wrapper class for each possible API call. The logic to invoke a tool is therefore bound to a single implementing class. This class must implement the logic to invoke the corresponding API. The class therefore must know the API path as well as the HTTP method required to invoke the API. It also must implement the request preparation and the handling of the response.
Workflow components are used within the actual workflow. These components have a sophisticated interface. Workflow components within a defined workflow phase are all invoked with the same parameters. Each workflow component generally consists of public methods to *invoke* and render which utilise the methods defined in the API wrapper. The invoking workflow therefore does not require to know which concrete workflow component is invoked. Workflow components instantiate API wrappers to conduct the actual API call. One workflow component is mapped to exactly one API wrapper. A workflow component therefore must be implemented for each tool that is part of the workflow. These components are also used for parallelisation within the tool-chain.
Workflow builder is the main component responsible for the creation of the actual workflow. The actual workflow is dependent on the configuration of client. The client has a JSON based configuration file (see Listing 3.3). The configuration file determines whether certain tools shall be run. Based on this configuration, the workflow builder instantiates the corresponding workflow components and adds them to the corresponding phase. The configuration determines the location of the API, as well as excluded domains. This information is propagated to the workflow components to provide the base service location for the API wrappers. Excluded domains are also passed to workflow components to define the scope just in time. After the workflow is set up, a dictionary of phases mapped to their workflow components is returned. By this logic, workflow components of a certain phase must share a common interface to

[6]https://palletsprojects.com/p/jinja/
[7]https://getuikit.com/

allow for uniform invocation.

```
{
  "runNuclei" : false,
  "runNucleiOnDomains": true,
  "runNmap":true,
  "detectWafs":true,
  "runSubdomainScan": true,
  "runNikto": true,
  "takeScreenshots": false,
  "runRetireJs": true,
  "runXsser": false,
  "runXssStrike": false,
  "runSqlMap": true,
  "runDalfox": true,
  "runTakeoverScan": false,
  "scanCVEs":false,
  "runGobuster":false,
  "includeAllDomainsOfCompany":false,
  "checkDomainStatus": true,
  "minimalCveScore": 7,
  "serviceUrl": "http://127.0.0.1:1337",
  "exclusions":[
      "subdomain.domain.com"
      ]
}
```

Listing 3.3: Example configuration file of the client

The **main entry point** is the class invoked by the user. Since this client is invoked from the command line, it offers the possibility to define targets and configuration utilising command line parameters. The following command line parameters are available:

- –config
- –domain
- –cookie
- –urlFile

The `-config` parameter defines a path to a configuration file which must be in a valid JSON format and contain the necessary keys to be interpreted correctly.

`-config` is an optional parameter, when this parameter is not supplied, the default configuration `config/config.json` is used.
`-domain` determines the actual target for the scan. Either `-domain` or `-urlFile` must be supplied for a scan to be started. If neither parameter is supplied, the scan is aborted.
`-cookie` is an optional parameter that sets the cookie which should be used for authorisation. If this parameter is not supplied, no cookie is used and all requests are performed in an unauthenticated manner.
In contrast to `-domain`, which takes a single domain as input, `-urlFile` can be utilised to provide a file containing multiple domains separated by line breaks. Several targets may thus be supplied by the use of this parameter.
After the command line parameters are interpreted the main entry point utilises the workflow builder to define the actual workflow. Once the actual workflow is built, the workflow components of each phase are invoked. Once the execution of each workflow component is finished, the main entry point tells the workflow component to render the results to a specific directory named like the actual target. After the workflow is finished and all results are rendered, the main entry point creates an `index.html` file which links all created result files along with a brief description.

3.5.2 Angular Frontend

The second client implemented throughout this prototype is implemented using Angular. In contrast to the Python approach, this client is interactive. Scan results are presented in real time to the user. However, this client is implemented in a stateless manner. Therefore, once the scan is started the client must be open the entire time to aggregate all results. Once the client is closed, all scans are aborted since the whole API interaction must happen on the client side. The configuration of the client can be done by activating several checkboxes. This client is suitable for small targets which can be scanned in a short amount of time. However, since the client must stay connected while the scan runs, this can become problematic on long-running scans. During the testing of this client it was observed that its interactivity degrades with the amount of time it has remained open. Another major drawback of this approach is the limited possibility of parallelisation. Browsers limit the maximum amount of operations that can be executed side-by-side. Scans using this client therefore tend to take longer than scans performed by the Python client since Python offers rich support for parallelisation.

3.5.3 Software Design

The client code of this prototype is implemented in TypeScript, which offers rich support for the use of interfaces and object-orientated programming. With this approach in mind, this client is implemented in an object-orientated manner. This section does not dissect the whole software architecture of TypeScript, nor does it discuss the general handling of components and dependencies. It rather focuses on the custom implementation while leaving general TypeScript specifics aside.
Two types of components are distinguished in this prototype:

- Tool-Components
- Main-Component

Components are a combination of TypeScript and HTML files which are coupled to one another. The TypeScript component refers the HTML component using the `@Component` directive. The`@Component` directive contains a `selector` which is used to refer to this specific component in another HTML component as well as a `templateUrl` field which defines the path to the corresponding HTML template. The HTML component is solely responsible for the interaction with the TypeScript component as well as rendering results. In contrast, the TypeScript component is used to communicate with external services, implement business logic and providing results which the HTML component can render.
Tool-Components are implemented for the invocation of a certain tool. TypeScript components are responsible for preparing the actual API call, as well as filtering and transforming received responses. The HTML components define how the results of the tools shall be rendered. All Tool-Components are decoupled from one another. Components used for the scanning of web applications follow a common interface to decouple the concrete implementation from the workflow. The rendering of results by the HTML component does not require reloading of the page. Rendering occurs in real time and without required user interaction.
The **Main-Component** is responsible for defining the actual user interface. This component aggregates all *Tool-Components* into a combined user interface. The main user interface relies on the usage of in-page tabs and drop-down pages. This component is also required to collect the user input defining the target, authentication method and configuration. The TypeScript component is required to define the actual workflow it instantiates all the *Tool-Components* and provides the necessary data for each individual component to perform its work. Due to the component-based architecture, the main component does not process any results of the *Tool-Components* since the rendering happens solely via the *Tool-Component*.

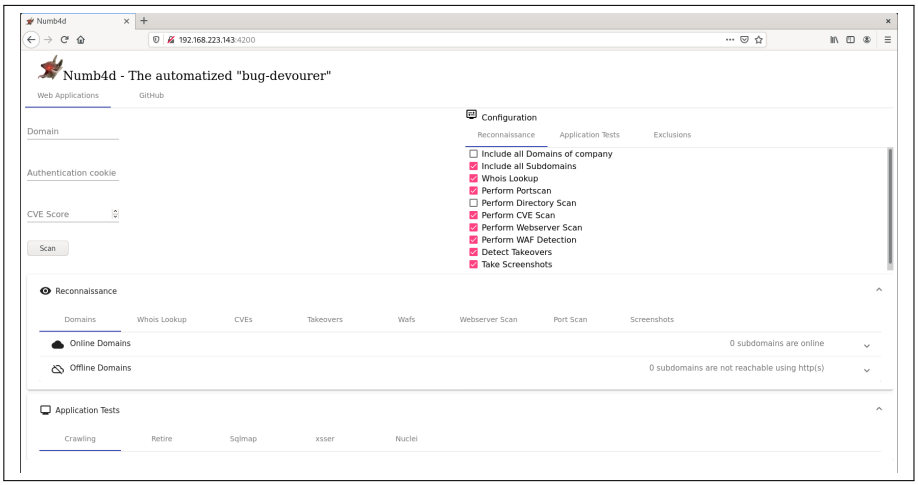

Figure 3.3: Appearance of the Angular prototype.

Figure 3.3 depicts the appearance of the Angular-based client prototype.

3.6 Toolchain

3.6.1 Tools

To achieve satisfying results, it is necessary to utilise a broad variety of tools. All the integrated tools are free to use and are mainly open source projects hosted on Github[8]. The primary focus of this prototype is to focus on web application tests as well as infrastructure-testing components. Well known tools which are used during web application penetration tests are therefore utilised, as well as a selected amount of tools which are used for infrastructure analysis. Each of the implemented tools must be dynamically invoked, so each tool has its own API method on the application server (see section A.1).

The implemented workflow is the same for each client. Tuning to the workflow such as skipping certain tools may be done by the client specific configuration. The workflow is depicted in Figure 3.4. The workflow is divided into four phases.

The first phase is the reconnaissance phase. This phase is responsible for gathering information on the target. It first utilises Whoxy (see 3.6.1.17) with the given input and attempts to enumerate all domains which are registered by the same company as the original domain. If Whoxy successfully retrieved all registered domains, it passes its result to Sublist3r (see 3.6.1.15). If Whoxy was not utilised, the original domain is forwarded to Sublist3r. Sublist3r is responsible for enumerating all subdomains of the supplied domains. The found subdomains are than passed to a tool which checks if the

[8]https://github.com/

domains are reachable using http or https. Domains reachable via HTTP or HTTPS as well as domains are not reachable via HTTP or HTTPS are considered online and are forwarded to nmap (see 3.6.1.13) and Netlas (see 3.6.1.16). Nmap performs the scan for the top 1024 ports on each of the domains also enumerating possible vulnerabilities based on the discovered services. Netlas is used to enumerate vulnerabilities that may still be present, as well as historic vulnerabilities. Domains reachable via HTTP or HTTPS are also forwarded to WafW00f (see 3.6.1.8), gowittness (see 3.6.1.7), Gobuster (see 3.6.1.9) and takeover (see 3.6.1.14). Wafw00f analyses which WAF solutions are employed or if no WAF is present. gowittness is utilised to take screenshots of the web pages which can be used to document the current state of the application or may be used to identify the used technology by, for example, default landing pages. Gobuster can be employed to discover unlinked files or directories by employing forced browsing. Takeover analyses the provided domain for possible domain takeovers based on the response of the application.

The second phase is domain testing. This phase aggregates all tools which are run solely against a single domain with the aim of actively testing it. Unlike the tools aggregated in the reconnaissance phase, tools in this phase actively scan for vulnerabilities tied to a single domain rather than gathering information about the target domains. Currently, only one tool is used in this phase. Nuclei (see 3.6.1.12) tests for a wide variety of vulnerabilities on the domain itself. Nuclei is only run against domains which are considered online by the online checking tool in the reconnaissance phase.

The third phase is the web application testing phase. This phase aggregates all tools which actively test web applications for various types of vulnerabilities. Only domains which are considered online by the online checking tool in the reconnaissance phase are processed. The first and most important step in this phase is the crawling of the web application. The crawling is performed by Gospider (see 3.6.1.6). The crawling process discovers files and directories on the webpage which are linked in the application itself, so no forced browsing technique is employed. Pages which offer several actions such as providing input can therefore be discovered. Gospider distinguishes between actual URLs and JavaScript files. JavaScript files discovered by Gospider are propagated to retire.js (see 3.6.1.11). Retire.js checks the given JavaScript files for common frameworks and checks for vulnerabilities in this specific version of the framework. URLs discovered by gospider are passed to the formAnalyser (see 3.6.1.4), as well as Dalfox (see 3.6.1.3) and sqlmap (see 3.6.1.5). Dalfox tests the supplied URLs for valid XSS injection points. Once injection points are detected, Dalfox actively fuzzes the injection points with predefined XSS payloads and analyses the response. formAnalyser is a helper tool for other tools which either require marked injection points

or have a poor content discovery functionality. As the name indicates, formAnalyser checks whether forms are present on the supplied page. If forms are present, formAnalyser creates a sample request based on the formAnalyser and fills the form data with the defined injection marker. After formAnalyser is finished, it passes its results to XSSer (see 3.6.1.1), XSStrike (see 3.6.1.2) and sqlmap. XSSer is used to test for XSS vulnerabilities in the supplied request while taking injection points into account. XSSer is another XSS detection framework which fuzzes the provided requests with XSS vectors while taking the marked injection points into account. sqlmap tests the input parameters for SQLi vulnerabilities. sqlmap not only checks for SQLi vulnerabilities in the supplied path, but also takes the plain results of gospider into account and tests all provided path parameters for possible SQLi vulnerabilities.
The fourth and final phase is for employing additional system tests. This phase only processes domains which are considered as online by the online checking tool of the reconnaissance phase. This phase consists of one tool. Nikto (see 3.6.1.10) checks the domain for common vulnerabilities. Nikto is run in this phase because it causes a large amount of requests to the domain which may result in further requests being blocked by the firewall or WAF. If blocks occur, they should occur at the end of the tests so that other tools are not influenced by them.

3.6.1.1 XSSer

XSSer is an automated framework capable of detecting and exploiting XSS-related vulnerabilities. It is open source and hosted on Github[9]. It is completely written in Python and requires Python in version 3.
In contrast to other applications, XSSer actively attempts to evade filters. It utilises over 1,300 attack vectors which are actively tested against the web application. Points for injection are required to be marked with *XSS*. XSSer refuses to actively fuzz all parameters of the target request. Attack vectors may be swapped or customised and are not hardcoded in the application. The presented attack vectors are actively fuzzed into GET and POST requests. XSSer follows the same pattern to detect XSS vulnerabilities as other frameworks. A request is sent with the fuzzed payload serving as a canary and the response is observed for said canary. If the canary is contained in the response, the injected payload is marked as valid and the sent request is reported.
Since XSS vulnerabilities are also likely to occur in an authenticated environment, XSSer offers the possibility to include session cookies in the fuzzed payloads.
During the testing phase, XSSer proved to be an outstanding tool in contrast to other XSS testing frameworks since it was able to detect stored XSS vulnerabilities. How-

[9]https://github.com/epsylon/xsser

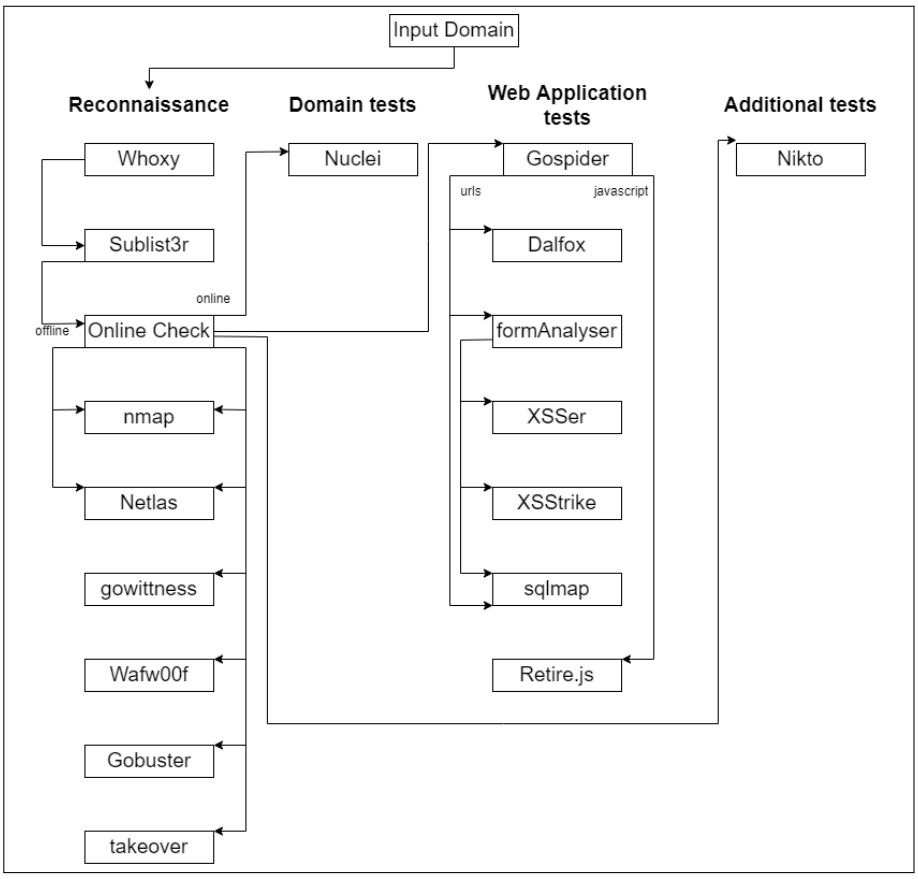

Figure 3.4: The tool based workflow of the prototype.

ever, during real-life test scenarios it became clear that XSSer is slow compared to other frameworks. Additionally, XSSer provides an outstanding number of false positives since detection mechanism is rather simple. Another drawback is that XSSer does not support crawling out of the box. Another tool thus must be utilised for detecting requests and forms which may be fuzzed. The substitution and identification of injection points is prone to errors in a real-life scenario.
Despite the drawbacks, XSSer was included in the prototype since it offers a large number of XSS attack vectors and can prove to be useful in a scenario with a small amount of in-scope items.

3.6.1.2 XSStrike

XSStrike is another framework capable of detecting XSS vulnerabilities. It is open source and hosted on Github[10]. It is written in Python and requires the Python 3.x runtime.
XSStrike does not attempt to evade filters and uses a payload generator for fuzzing. XSStrike does not require injection markers and is able to fuzz all possible inputs without the need to mark injection points. This makes XSStrike attractive for use in an automated toolchain. XSStrike also supports crawling and is therefore able to automatically identify forms and fuzz-able requests without further interaction or the help of another tool.
The XSS vulnerability verification is once again done using canaries in the request. If the response contains said canaries, the target is seen as vulnerable and the sent request is provided as a PoC.
XSStrike also offers session support and can send session cookies with each request.
XSStrike is not only able to detect reflected and stored XSS vulnerabilities, but also checks for Document Object Model (DOM) based vulnerabilities. The verification of DOM-based XSS vulnerabilities is rarely implemented in free-to-use XSS vulnerability detection framework since such vulnerabilities are uncommon and are hard to test and verify.
XSStrike is slow compared to other frameworks with the same capabilities. The amount of false positives is lower compared to XSSer, however, the amount of false negatives is also higher. XSStrike offers a crawling feature which does not work as intended and is not usable in a real-life scenario. The crawler omits all input fields which are not explicitly <*input*> elements. However, tags such as <*textarea*> are commonly used for forms and such tags would not be detected. Another drawback is that output

[10]https://github.com/s0md3v/XSStrike

colours cannot be disabled out of the box. The inclusion of Unix colour codes eliminates the possibility to parse the output of the tool elegantly. It is necessary to disable the colours directly in the source code.
XSStrike was included in the prototype since it verifies DOM-based XSS vulnerabilities. The unreliable crawling feature can be disabled, and XSStrike can be chained with other, more reliable solutions to discover content. Therefore, XSStrike may be used as a second XSS discovery framework in niche cases with a small amount of in-scope items.

3.6.1.3 Dalfox

Dalfox is an open source XSS scanning tool. It is written in go and hosted on Github [11]. Dalfox does not require a whole go development environment. Packed releases can be executed without the whole installation of go.
Dalfox is able to discover different XSS attacks. It features the detection of stored, DOM based as well as reflected XSS vulnerabilities. It detects vulnerabilities by integrating canaries in the request which must be reflected in the response. What makes Dalfox outstanding is its performance. During testing, it proved to be faster than other evaluated XSS detection frameworks.
Dalfox actively fuzzes data in POST and GET requests. It does not simply inject `<script>` tags in the payload. It features attack vectors and bypass techniques such as event handlers. Furthermore, it is capable of parameter mining and parameter analysis and is thus not required to be coupled with an external tool to retrieve relevant data required for the respective request. The capabilities of Dalfox are not limited to pure XSS detection. It features a feature called "Basically another vulnerability" which is able to detect other vulnerabilities such as SQLi, Server Side Template Injection (SSTI) and Open Redirect.
As noted, Dalfox is much faster than the other XSS detection frameworks. It does not require marked injection points since it actively fuzzes all request parameters with a canary. When a potential XSS vulnerability is detected, Dalfox provides a PoC URL. However, the detection of stored XSS vulnerabilities requires a server which is running and contacted by the payload. Stored XSS vulnerabilities may be discovered by running Dalfox in `sxss` mode which starts up a server. The detection of stored XSS vulnerabilities is not integrated in this prototype since it would consume too many resources by starting a server for each scan.
Dalfox is used as the main scanner for XSS vulnerabilities in this thesis. If the detec-

[11] https://github.com/hahwul/dalfox

tion of stored XSS vulnerabilities is desired, another tool such as XSSer may be used along with dalfox.

3.6.1.4 formAnalyser

formAnalyser is a custom developed tool for this prototype.
Since many tools rely on whole URLs as their input and lack the capability of detecting inputs on a webpage on their own, a tool is required to fill this gap. formAnalyser is a simple Python-based tool which discovers forms on webpages. It is capable of detecting forms and building a sample request string based on the method and target on the form. It may also be used to inject custom payloads in the input elements of the form since some tools require a certain injection string to evaluate where payloads shall be placed.

3.6.1.5 sqlmap

sqlmap is an open source tool capable of detecting and exploiting SQLi vulnerabilities. It is written in Python and requires the Python 3 runtime. The source code is publicly available on Github[12].
The manual detection and exploitation of SQLi vulnerabilities is difficult. The sheer amount of different SQLi vulnerabilities cannot be tested by hand. Non-blind SQLi vulnerabilities may be easily detected and exploited by hand, however, blind SQLi are often dependent on aspects such as the time required to process the tampered query. sqlmap aims to address these obstacles by providing a powerful framework which is capable of detecting different kinds of SQLi vulnerabilities, as well as supporting the exploitation of various Database Management Systems.
sqlmap supports the detection of databases, tables and the exploitation of various systems based on the used DBMS. sqlmap automatically fuzzes the provided URL or request with payloads typical for SQLi attacks. In the first iteration, sqlmap fuzzes all parameters with payloads for different DBMSs. Based on the response of the target, sqlmap detects whether the target is vulnerable to SQLi attacks. If the target is vulnerable, sqlmap determines which DBMS is used based on the used payloads. Once the DBMS is detected, sqlmap only uses payloads tailored for this system to exploit the vulnerability. Blind types of SQLi can be automatically detected since sqlmap takes different aspects (e.g. the time needed to process the query) into account when verifying the vulnerability.
sqlmap integrates with other tools used during penetration testing. As example in-

[12]https://github.com/sqlmapproject/sqlmap

tegration with metasploit[13], OWASP Zed Attack Proxy (ZAP)[14] and BurpSuite[15] is possible. However, in this prototype, sqlmap is not coupled with any external tool since the detection of SQLi vulnerabilities is required. The prototype does not force sqlmap to exploit these vulnerabilities. Actual exploitation is done by the individual using the prototype. sqlmap offers the switch `-batch` which runs sqlmap without user interaction. However, this approach can may yield an increased amount of false positives since sqlmap automatically runs queries for the first DBMS it detects. The first detected DBMS is often not the correct one, which leads to sqlmap sending requests tailored to the wrong DBMS.
Since SQLi vulnerabilities may occur in an authenticated environment, sqlmap is able to use authentication or sessions based on user input. The framework is also able to detect forms and injection points. However, since the crawling feature proved to be unreliable the tool is paired with the external formAnalyser tool to detect forms and inputs. Queries and requests detected by formAnalyser are passed to sqlmap which performs the checks for SQLi vulnerabilities.
Because sqlmap provides a low amount of false positives the tool is described as reliable and is the main tool used for the detection of SQLi vulnerabilities. In this prototype, sqlmap is run with the `-batch` operator to enable a server side execution where no user interaction is required. While this increases the amount of false positives, it is the only option to integrate the tool in this prototype since no user interaction can occur on the server side.

3.6.1.6 Gospider

Gospider is an open source crawler capable of crawling websites and discovering their content. As the name indicates, Gospider is written in go. The source code is publicly available on Github[16].
Crawling is a crucial part of the analysis of the web application. The crawling phase involves following links, submitting forms and detecting resources such as CSS and Javascript (JS) files. Gospider is an elegant approach to web application crawling capable of following links throughout the web application, submitting forms, detecting JS files and providing the respective discovered paths. Gospider has a built-in mechanism to avoid endless loops. Gospider also parses the sitemap.xml as well as the robots.txt file and adds the found paths to the result.
In addition, Gospider is able to find AWS-S3 from response sources, enumerate subdo-

[13]https://www.metasploit.com/
[14]https://www.zaproxy.org/getting-started/
[15]https://portswigger.net/burp
[16]https://github.com/jaeles-project/gospider

mains based on links on the webpage and fetch URLs from sources such as Wayback Machine and Virus total. Gospider also utilises a random User-Agent header to avoid blocks by the web application.
The support for sessions is also built-in, paths in an authenticated environment may therefore also be discovered by the crawler. The output of the application is easy to parse since it does not enable colours per default and adds annotations for each discovered URL and JS file. The server method responsible for invoking Gospider can therefore distinguish between URLs and JS files and provide these results in a separated form.
During the testing phase, Gospider was the fasted crawler. It was able to crawl the application in a short amount of time while providing the results in a structured form. The results of Gospider are the foundation for fuzzers such as XSSer and sqlmap.

3.6.1.7 gowitness

gowittness is an open source screenshotting utility written in go. The source code is publicly available on Github[17].
During the penetration testing of web applications screenshots are an important element. Screenshots can easily be analysed and may contain sensitive data such as default server landing pages. Screenshots can therefore be used either to detect the technology used by the server or to document the current appearance of the web application.
gowittness utilises a headless Chrome browser to open the provided web applications and screenshot them. Since the browser operates in a headless manner, no user interface is required on the server. gowittness may operate on single URLs or whole subnet masks. It maintains a database consisting of screenshots and browsed webpages. It also offers the possibility to run a dedicated server which displays the screenshots.
In the context of this prototype, the tool is utilised to browse the default landing pages of each single domain. It does not take screenshots of subpages detected by a crawler. Screenshots are not presented using the built-in server since the screenshots are base64 encoded and are transmitted to the client in this form.

3.6.1.8 Wafw00f

Wafw00f is an open source tool used for the detection of WAFs. It is written in Python and requires the Python 3 runtime. The source code is publicly available on Github[18].
A WAF is employed to protect a web application against HTTP-based attack vectors.

[17] https://github.com/sensepost/gowitness
[18] https://github.com/EnableSecurity/wafw00f

In contrast to a normal firewall a WAF inspects the incoming and outgoing traffic on the application layer. A WAF is thus able to detect attack vectors before they reach the web application. The WAFs of different vendors often have a type of fingerprint based on the way they interact with malicious traffic. Therefore, wafw00f is able to fingerprint the vendor and WAF based on the behaviour when dealing with malicious traffic.
In the first step, Wafw00f sends a number of normal requests to the web application. Based on the responses, Wafw00f determines a baseline. In the next step Wafw00f sends a number of malicious requests to the web application. The responses of such malicious requests are analysed and interpreted. Based on the analysis, the actual WAF may be determined.
The detection of the actual WAF may prove to be useful since different tools offer the possibility to bypass WAFs by tampering requests so that the WAF does not detect the malicious activity. The absence of a WAF means that the attack vector is increased since no additional security measures are employed on a the application level.
wafw00f limits its number of requests and is therefore quite fast. The detection of the actual WAF becomes increasingly useful in the later exploitation of vulnerabilities.

3.6.1.9 Gobuster

Gobuster is an open source brute-forcing tool which can be utilised for several use cases. Gobuster is written in go and the source code is publicly available on Github[19]. Gobuster offers a plethora of features related to brute-forcing:

- Directory and file brute-forcing
- DNS subdomain brute-forcing
- Virtual host enumeration
- Amazon S3 buckets enumeration

The most important aspect of Gobuster in the context of this prototype is the support for directory and file brute-forcing. This type of attack allows the attacker to enumerate directories and files on the target server with the use of word lists. Such files and directories may not be directly linked on the webpage and therefore cannot be detected by a crawler. Such an attack is also referred to as forced browsing. With the use of this technique, hidden paths such as administrative interfaces or backup files may be discovered on the target. Gobuster provides the possibility to include custom word

[19]https://github.com/OJ/gobuster

lists. To discover as many paths as possible, an extensive word list as included in seclists[20] may be used. The disadvantage of using large word lists is the sheer amount of requests which may result in the used IP address being blocked by the web application. To detect paths, Gobuster makes use of the returned status codes. By default, the status codes 200 OK, 401 Unauthorized, 403 Forbidden and 500 Internal Server Error are interpreted as findings. Paths returning 404 Not Found are ignored since this status code implies that the respective path is not found on the server.
Gobuster is one of the fastest tools for forced browsing. Compared to tools such as DirBuster, this tool stands out with its performance and flexibility. Another major benefit of Gobuster is that no user interface is included. Instead, this tool solely relies on the command line, which makes the output easy to parse by an external tool.
Although Gobuster is an outstanding tool, its use should be limited. During testing, it became clear, that web applications tend to block IP addresses after a large number of successive requests. Other tools such as Nuclei are also able to detect unlinked administrative interfaces.

3.6.1.10 Nikto

Nikto is an open source web server scanner written in Perl. The source code of Nikto is publicly available on Github[21].
Nikto is a tool which has been around for a long time. This tool still proves useful for the analysis of web servers. It performs comprehensive tests against web servers while testing attack vectors, including the detection of programs and files, the detection of outdated software and the analysis of server configuration. Nikto includes checks for over 1,250 web server versions. (Sullo, 2018) It also analyses over 270 configuration flaws specific to server versions. (ibid.)
However, Nikto is not designed to be stealthy. It takes turns to test a web server as fast as possible, which results in a large amount of successive requests which may lead to the client IP being blocked by the web application.
Nikto offers different formats for its output. This facilitates the integration into different workflows. The Nikto findings can be saved as a JSON file, which allows for easy parsing by the workflow which originally invoked Nikto.
Nikto is used in this prototype to cover several aspects of the web server analysis which other tools cannot fulfil. Nikto detects headers which other tools would not report but which prove to be relevant for further investigation. Moreover, the tailored checks for several server versions prove to be useful and are not covered by any other tool to

[20] https://github.com/danielmiessler/SecLists
[21] https://github.com/sullo/nikto/

this extent. Although Nikto tries to scan the target as quickly as possible, it is still slow compared to modern tools. Since Nikto executes a large amount of requests, it is deployed at the end of the actual workflow in order to avoid IP blocks.

3.6.1.11 retire.js

retire.js is an open source tool which is capable of detecting outdated JavaScript dependencies which are used throughout a web application. The original retire.js tool is written in JavaScript and is available on Github[22]. However, since this prototype mainly relies on Python scripts on the server side, a Python port of retire.js which is also available on Github[23] is used in this prototype.
Many web applications rely on specific versions of JavaScript libraries such as jQuery in order to enhance their interoperability and front-end behaviour. However, these dependencies are often overlooked in the updating process. Some dependencies must be in a specific outdated version since new versions may remove or alter existing features which may cause breaking changes or misbehaviour of the web application which uses it. Since most of these dependencies are open source researchers look at such libraries and tend to find exploitable functions in them which lead to vulnerabilities. Only some functions of such a library are thus vulnerable. Web applications which use such outdated libraries may not automatically be at risk since they do not have to use the vulnerable function. However, the use of outdated and vulnerable libraries offers an increased attack vector.
Outdated libraries are not a valid finding in many bug bounty programs since the application is not automatically vulnerable. However, retire.js may still be used to find possible vulnerabilities. The retire.js component in this prototype relies on the crawler to provide all found JavaScript files since retire.js is solely used to analyse the dependencies and has no dedicated crawling feature.

3.6.1.12 Nuclei

Nuclei is a general use vulnerability scanning tool capable of identifying different types of vulnerabilities. It is programmed in go and is available on Github[24].
Nuclei is a tool that differs from other vulnerability scanning frameworks. Instead of relying on certain pre-set attack vectors, it is solely template driven. The templates utilised by Nuclei are hosted on a separate Github repository[25]. Nuclei runs

[22]https://github.com/retirejs/retire.js/
[23]https://github.com/FallibleInc/retirejslib
[24]https://github.com/projectdiscovery/nuclei
[25]https://github.com/projectdiscovery/nuclei-templates

these templates against the defined target and uses the `matchers` in combination with the `path` parameter of the template to detect whether the vulnerability is present. If such vulnerability is present, it utilises the `extractor` parameter to potentially extract sensitive data such as usernames. In comparison to other vulnerability detection frameworks nuclei is not limited to a certain vulnerability detection purpose. It can be utilised as a general use vulnerability scanner since its capabilities are only driven by the used templates. Templates are generally identified by tags. Groups of templates may be included or excluded based on their respective tag.
Nuclei ships with a plethora of available templates which cover different kinds of vulnerabilities. According to Projectdiscovery, 2022 the top 10 most commonly used templates are:

- Templates for the discovery of over 1,000 known CVEs (Tag: cve)
- Templates for the discovery of administrative panels (Tag: panel)
- Templates for the identification of local file inclusion (Tag: lfi)
- Templates for discovering XSS vulnerabilities (Tag: xss)
- Templates for discovering vulnerabilities relating to WordPress (Tag: WordPress)
- Templates for discovering RCE vulnerabilities (Tag: rce)
- Templates for identifying and extracting sensitive data (Tag: exposure)
- Templates for discovering CVEs published in 2021 (Tag: cve2021)
- Templates for identifying vulnerable WordPress plugins (Tag: wp-plugin)
- Templates addressing the general technology stack of the target (Tag: tech)

Since each template of Nuclei requires a respective request which leads to a great deal of traffic, if many templates are used. Nuclei employs the strategy of clustering templates based on their respective `path` parameter. If several templates point to the same path, only one request is sent and analysed by the respective `matchers` parameters.
Nuclei is able to present its results in different formats which can be read and interpreted by other tools, which eases the integration into a toolchain. In the context of this prototype, the output of nuclei is presented in JSON and later deserialised.
Nuclei serves as one of the core components of this prototype since it is able to detect a wide variety of vulnerabilities while maintaining a generally low scan time compared to other tools. Since Nuclei is template-driven, the amount of false positives is low. Nuclei is run against every domain in scope and not run against every single sub site

on the domain since the paths of the templates are built to be run against the whole domain in most cases.
The prototype only utilises selected templates since many templates, like the lack of security headers, are considered out of scope in many bug bounty programs.

3.6.1.13 nmap

nmap is an open source tool used to enumerate hosts in a network and discovering open ports. nmap is written in C++ and is available on Github[26].
nmap is a well-known tool for the reconnaissance phase of an engagement. It is capable of enumerating open ports and services of various hosts. nmap is highly scalable based on the infrastructure and hosts in scope. It can easily be extended with Nnmap Scripting Engine (NSE) scripts. nmap comes with a plethora of different scripts which are able to detect different aspects of the target. Among other functionalities, nmap can detect ports, services, specific service versions and operating system of the host. Port scans are featured for both Transmission Control Protocol (TCP) and User Datagram Protocol (UDP). However, UDP scans are much slower and require root privileges on the system executing nmap. nmap offers various modes of scanning which are more or less stealthy. Less stealthy modes are faster compared to stealthy modes, although they are more likely to be discovered by a firewall or Intrusion Detection System (IDS). nmap may scan all ports on the target system, however, the method of scanning only the 1,024 most common ports is a more promising approach since most targets do not change their port number for standard services and scanning of only 1,024 ports is much faster and less likely to be discovered. nmap offers the possibility to present its findings in the Extensible Markup Language (XML) format, which makes parsing of actual results by third-party software easier.
In this prototype nmap is utilised to detect services of the targets. The scan consists of ports and services, as well as their versions and operating system. Only the top 1,024 ports are scanned in order to make the reconnaissance phase of the prototype faster while maintaining viable results. UDP scans are omitted, and TCP scans are executed. The output is provided in XML format which makes the parsing by the actual implementation easier and more accurate. However, the XML output is not deterministic since the type of XML tags may vary based on the findings. All targets for scanning are assumed to be online.

[26]https://github.com/nmap/nmap

3.6.1.14 takeover

takeover is an open source tool used scan domains for possible domain takeover vulnerabilities. It is written in Python and available on Github[27].
takeover takes an input of domains and subdomains. It takes turns to request the landing page of each provided domain. Based on the result of the request of the provided URL, it matches the response to the default responses of different providers if the given domain name is no longer registered by that provider. takeover currently checks for domain takeovers of over 50 providers[28]. For a domain takeover to be applicable, it must be registered on a provider which offers the possibility to choose a domain name while setting up the service. For example, a domain may be hooked to a backend service run by Surge. If the registrant does not renew the backend service the provider takes turns and deactivates the backend service. An attacker may therefore be able to register said backend service on, for example, Surge. After the successful setup process, the original domain will still access the backend service which is now controlled by the attacker, leading to the attacker being able to supply potentially malicious source code on a trusted subdomain of a company.
The prototype employs takeover since it is able to detect potential takeovers for a wide variety of providers, as well as act in a fast manner since the detection is solely error based. Domain takeovers are a highly rated vulnerability.

3.6.1.15 Sublist3r

Sublist3r is an open source tool used to enumerate subdomains. It is written in Python and the source code is available on Github[29].
Sublist3r is a tool capable of enumerating subdomains based on different sources such as

- Google
- Yahoo
- Baidu
- Ask
- Netcraft
- Virustotal

[27]https://github.com/m4ll0k/takeover
[28]https://github.com/m4ll0k/takeover/blob/master/takeover.py
[29]https://github.com/aboul3la/Sublist3r

- ThreatCrowd
- DNSdumpster
- ReverseDNS

In contrast to other subdomain enumerating tools, it relies on different sources as mentioned above and does not require a wordlist to enumerate subdomains. Since no wordlists are required, it is more likely to detect actual subdomains. These domains must not be contained in a wordlist as they are fetched from search engines or historic data samples.
Sublist3r operates in a highly performant way, it is therefore faster and more efficient than other comparable subdomain enumeration tools while still maintaining low amount of false negatives. One downside of Sublist3r is the lack of an option to disable output colours. Output is formatted employing the UNIX colour scheme, which makes parsing of the output almost impossible. A script is therefore run during the installation to alter the source code in a manner which disables colours.
Virustotal also tends to block requests, so this resource is likely to not be available during many successive invocations of the tool which may alter the findings slightly.
This tool is a core component throughout this prototype. It is executed prior to any other tools if the given bug bounty program has subdomains in scope. It is solely used to enumerate the subdomains and does not make assumptions as to whether the domains are reachable.

3.6.1.16 Netlas

Netlas is an open source and partly free-to-use internet scanner available on https://netlas.io/. It also features a Python based Software Development Kit (SDK) which enables Netlas to be used without the need to build ad hoc queries and fill them into the user interface.
Netlas scans every IPv4 address and crawls found web applications employing different protocols. It enhances the results with additional data such as protocol headers and disclosed versions. Since it scans every IPv4 address, it is able to find different vulnerabilities on these addresses. The results are stored in Netlas database and updated during successive scans. With the latest version, Netlas also discovers DNS records and performs reverse DNS lookups to identify companies and registrants coupled to different domains. The web interface of Netlas features a powerful elastic search engine. A researcher is therefore able to build dynamic queries against the Netlas database.
Currently, Netlas is not popular. However, its features and possibilities of integration are promising. In the future, Netlas could become a competitor of other well known

internet-scanners such as Shodan[30]. The Netlas developers are keen to promote their product. I was therefore granted an unlimited amount of searches every day to evaluate the usability of Netlas and built it into this prototype.
The results of Netlas are outstanding. It discovers possible vulnerabilities and links them to known vulnerabilities. However, such results always be taken with a grain of salt. Netlas does not necessarily update its database when services are updated or removed, therefore Netlas tends to deliver a high amount of false positives. The output of Netlas should thus be verified and not immediately reported to a bug bounty platform.
The prototype uses a self-made script which employs the Python SDK. Therefore, no interaction with the actual user interface of Netlas is required. Although Netlas tends to deliver a high amount of false positives, it is still a valid addition to the other employed tools since the researcher may be able to leverage (historic) details which would not be disclosed by other tools.

3.6.1.17 Whoxy

Whoxy is an API capable of performing Whois and reverse Whois queries. Whoxy is not free to use, however it is inexpensive compared to other APIs performing Whois queries.
Whois data may include sensitive details such as the registrant, company and contact information of an individual who registered a public domain. Often this information is stripped and replaced with the data of a company which registers domains for clients. If this is not the case, a researcher may benefit from the disclosed data.
The use of Whoxy is not only limited to the enumeration of Whois data since it is also able to perform reverse Whois queries. The term 'reverse Whois query' refers to the process of enumerating domains based on the registrant. Many bug bounty programs limit the scope to all public domains of the client rather than to selected domains. In this case, Whoxy can be utilised to enumerate all domains registered by the client.
Whoxy is used in this prototype in cases where all public domains are considered to be in scope. Simple Whois data may also be included, however, in the context of bug bounty hunting, this should not be included without reverse Whois queries since plain Whois data may not point to actual security misconfigurations.
Compared to other Whois data providers, Whoxy is the cheapest solution that still maintains viable results. 1,000 Whois queries cost $2 while 1,000 reverse Whois queries cost $10 as of June 2022.
Since Whoxy offers a simple JSON-based API, it is easily integrated into an automatic process.

[30]https://www.shodan.io/

3.6.1.18 Trufflehog

Trufflehog is an open source tool capable of discovering sensitive data in different git based repositories. It is written in go and is available on Github[31].
Trufflehog is able to detect sensitive information such as connection strings, usernames, passwords and private keys in git-based repositories. Trufflehog scans every commit in the repository, so old commits which may have included sensitive information that was later removed are still discoverable. It is necessary to clone the repository onto the local file system for the actual scan. Trufflehog therefore must be coupled with git command line interaction to work on the server invoking the tool.
In rare occasions bug bounty programs consider git repositories to be in scope. On these occasions the tool prototype can be utilised to perform an analysis of these repositories using Trufflehog. This functionality is not integrated in the automated workflow of the prototype and must be invoked as a standalone functionality.

3.7 Coverage of the OWASP Top 10

The OWASP Top 10 describes the top 10 causes for vulnerabilities in web applications (see section 2.3). It is thus necessary to cover as many of these risk factors as possible. This section focuses on the risks described in section 2.3 and states how they can be covered by the prototype. If a certain criteria is not covered by the prototype, it is evaluated why it cannot be fully covered.

3.7.1 A01:2021 – Broken Access Control

Certain types of vulnerabilities in this category cannot be detected by automated approaches. For example, privilege escalation cannot be easily detected since this prototype lacks the capability to authenticate on its target and cannot fully cover the privilege principal implemented by the application. Missing access control cannot be fully detected since the prototype is not aware which pages would need additional protection, as this is heavily dependent on the context of the web application.
However, certain types of vulnerabilities can be detected by this vulnerability. For example, forced browsing to the *server-status* page of Apache, which holds sensitive information and is protected by most implementations, can be detected by the *gobuster* tool. The *nuclei* tool is able to detect whether an application suffers from vulnerabilities regarding broken access control.

[31] https://github.com/trufflesecurity/trufflehog

3.7.2 A02:2021 – Cryptographic Failures

This prototype does not focus on cryptographic failures. It is difficult for an automated approach to detect the randomness of, for example, cookies of a target, since this would lead to an increased number of requests and increased resource consumption. Additionally, many bug bounty hunting programs consider vulnerabilities such as weak cryptographic ciphers to be out of scope. However, the prototype could be enhanced with a tool to analyse certain cryptographic failures. This was not included in the first implementation.

3.7.3 A03:2021 – Injection

The prototype covers many injection types. This OWASP category includes vulnerabilities related to SQLi, XSS and RCE. The prototype utilises *sqlmap* to discover SQLi vulnerabilities. *Dalfox*, *XSStrike* and *XSSer* can be used to discover DOM based, reflected and stored XSS vulnerabilities. The *nuclei* tool is able to discover RCE, XSS and SQLi vulnerabilities. However, this prototype lacks the detection of out-of-band interaction.

3.7.4 A04:2021 – Insecure Design

This prototype does not have the implementation to detect insecure designs of applications. However, it is able to detect secrets in the source code of open source applications using *tufflehog*. The disclosure of secrets in the source code is regarded as insecure design.

3.7.5 A05:2021 – Security Misconfiguration

The *nuclei* tool is able to discover vulnerabilities related to this OWASP category. Usage of outdated software is a part of this category. Outdated software can be detected by *nuclei* or *nmap*. This category also includes the lack of security headers and revelation of stack traces, which are often considered to be out of scope by bug bounty hunting programs.

3.7.6 A06:2021 – Vulnerable and Outdated Components

As mentioned above, the tools *nuclei* and *nmap* are able to discover outdated components. Based on the targets' configuration, *nmap* can also fingerprint the operating

system version of the target. Both *nuclei* and *nmap* are able to propose exploits based on the version number of the used components.

3.7.7 A07:2021 – Identification and Authentication Failures

This category is not covered by the prototype. The prototype is not able to detect vulnerabilities related to single sign-on and multilevel authentication. Additionally, the prototype cannot detect the reuse of session identifiers since it does not offer the possibility of authentication. This prototype does not implement mechanisms to brute-force user credentials.

3.7.8 A08:2021 – Software and Data Integrity Failures

This category is not covered by the prototype. Automation cannot detect the update and patching mechanisms which an application has in place. Additionally, the CI/CD pipelines are rarely available to the attacker.

3.7.9 A09:2021 – Security Logging and Monitoring Failures

The prototype is able to discover security issues regarding to logging and monitoring. The tool *nuclei* is able to detect logging frameworks. When a logging framework is detected it is possible to check whether the log files are unprotected. Additionally, the tool *nmap* is able to discover ports which are commonly used for monitoring, such as Simple Network Management Protocol (SNMP). When SNMP ports are opened, it is possible to see whether SNMP information is sufficiently protected.

3.7.10 A10:2021 – Server-Side Request Forgery (SSRF)

The tool *nuclei* is able to identify known SSRF vulnerabilities. However, the prototype lacks the ability to automatically fuzz payload which cause SSRF vulnerabilities or to automatically detect out-of-band interaction.

Chapter 4

Evaluation and Outlook

4.1 Evaluation of the Prototype

4.1.1 Technical Performance

In order to evaluate the prototype it was tested in various bug bounty hunting programs. The prototype was executed on a virtual private server hosted on OVH[1]. The server was scaled higher than necessary and consisted of a 8vCore processor, 16 GB of RAM and a Solid State Drive (SSD) with 160 GB memory. The server operating system was Debian 11. The server caused a monthly cost of $50. The virtual private server hosted the prototypes' server backend, client application and database. The applied architecture is therefore described as single-tier. The choice of architecture was not optimal for the prototype since it should consist of a database server, an application server and a client. Due to the chosen architecture, the server's Central Processing Unit (CPU) usage nearly at 100%. Additionally, the prototype serves the possibility of parallelisation. To speed up tests tools were executed with the following parallelisation settings:

- gospider: 1 thread
- Whois: 1 thread
- Netlas: 10 threads
- Dalfox: 5 threads each thread resulting in 100 workers on the server side
- Sublist3r: 1 thread

[1] https://ovh.de/

- Gowittness: 30 threads
- Retire.js: 30 threads
- sqlmap: 5 threads
- Takeover: 1 thread
- wafw00f: 1 thread
- XSStrike: 5 threads
- XSSer: 5 threads
- nmap: 10 threads
- nikto: 10 threads
- online checker: 1 thread
- Trufflehog: 1 thread

Tests were mainly conducted using the Python client implementation of this prototype since it proved more useful for longer-running tasks requiring minimal user interaction. The Python client does not implement the possibility to execute forced browsing by using `gobuster`. Automated forced browsing proved to be too invasive for the most bug bounty hunting programs often resulting in IP bans. Using these parallelisation settings, more CPU resources were used than RAM. The RAM utilisation peaked at approximately 8 GB. When using this prototype in a single-tier architecture, higher resources of the CPU are required rather than RAM resources.

4.1.2 Performance in Bug Bounty Hunting Programs

After implementation, this prototype was tested in several bug bounty hunting programs on different platforms. To evaluate its feasibility, it was tested in programs having all subdomains of a certain domain in scope. The number of domains ranged between 50 and 3,300.
The prototype was tested in nine programs on HackerOne and 11 vulnerabilities were reported. Table 4.1 lists the submitted vulnerabilities alongside their acceptance and programs. Details about the bug bounty hunting programs were redacted according to the disclosure policy of HackerOne. It is notable that only two out of eleven vulnerabilities were considered valid.
The vulnerabilities *Disclosed web.config*, *User enumeration*, *API key disclosure* and *Outdated SSH (6 years)* were not accepted by the triaging team since they do not clas-

sify these vulnerabilities as a security risk.
SSH User enumeration submitted for *Program 3* and *Program 4* were not accepted as a security vulnerability by the triage team. It is notable that the same vulnerability which was submitted 12 days earlier was accepted.
*WordPress user enumeration*s are described as a normal WordPress functionality by the triage team and do therefore not qualify as a security vulnerability in the context of bug bounty hunting. The vulnerability *Subdomain takeover* was not accepted since the provided PoC did not work as expected.
The prototype was also tested against three programs hosted by BugCrowd. Seven vulnerabilities were submitted to BugCrowd, four of which were accepted. Table 4.2 lists the vulnerabilities and their programs. According to the disclosure policy of Bugcrowd, details about the bug bounty hunting program were redacted.
The vulnerability *Disclosed WADL File* was not accepted by the triage team since it did not prove to be a relevant security risk. However, this vulnerability disclosed detailed information about the used API allowing an attacker to directly communicate with the API without scraping API information using an interception proxy. Additionally, certain API calls disclosed stack traces which disclosed information about the used Content Management System (CMS), as well as the backend technology. After informing the BugCrowd triage team about these additional risks, no response was given.
WAF bypass using WADL information is a direct result of *Disclosed WADL File*. In this scenario, an attacker leveraging the information from the WADL file is able to bypass the WAF filters. The web application responded differently when certain methods declared in the WADL file were called directly rather than being called from the context of the web application. In this case, the direct API calls were allowed rather than being forbidden when being called from the context of the web application. However, the BugCrowd triage team did not see any security impact arising from this vulnerability.
Two *Reflected XSS* vulnerabilities were considered invalid, since one of them is based on a HTTP-POST-request. The other causes XSS on an error page, which allows redirects which can be used for phishing. Since phishing is considered out of scope and HTTP-POST based requests are tricky to execute by an user, the triage team decided not to accept these vulnerabilities.
During the prototype testing an agreement with a Dutch university was established. This university offers a private bug bounty hunting program which prohibits the use of automated scanners. After contacting the university for permission to use this prototype in their bug bounty hunting program, explicit permission was given. Since this program otherwise prohibits the usage of automated scanners, this was a valid example

to see how the prototype would perform in a program where no other automated scanners had been used before. Seven vulnerabilities were discovered in this program, all of which were accepted. The vulnerability *User Enumeration (WordPress)* was submitted for three domains. During the tests in this program, close contact was held with the security team. The security team was able to submit firewall information which is useful for the evaluation of this prototype.
Figure 4.1 depicts the requests of the prototype which were registered by the firewall. The tests in this program were performed between June 13th 2022 and July 1st 2022. In total, 52,965,417 requests were performed by the prototype. Which means the prototype typically hits its target with an average of 36 requests per second. This is mainly caused by the missing rate limits of the prototype. Most requests were performed between June 17th and June 24th. During this timeframe large scaled fuzzing operations were performed to test for XSS- and SQLi-related vulnerabilities.
Figure 4.2 lists the request headers which were registered by the firewall. The most common request header was *Mozilla 5.0 (Macintosh; Intel Mac OSX 10.15; rv:75.0) Gecko/2010010 Firefox/75.0*, with 61.41% of requests. The second most common header was *sqlmap/1.6.5.5#dev (https://sqlmap.org)* with 37.28% of requests. The header *Mozilla 5.0 (Macintosh; Intel Mac OSX 10.15; rv:75.0) Gecko/2010010 Firefox/75.0* was sent by Dalfox during the automated testing for XSS-related vulnerabilities. The header *sqlmap/1.6.5.5#dev (https://sqlmap.org)* was sent by sqlmap during the automated tests for SQLi-related vulnerabilities.

Vulnerability	Program	Scope	Reported at	CVSS	Accepted
Disclosed web.config	Program 1	Webapplication	18.05.2022	3.7	No
User Enumeration	Program 2	Webapplication	11.03.2022	3.7	No
SSH User enumeration	Program 3	Webapplication	16.02.2022	6.5	No
SSH User enumeration	Program 4	Webapplication	26.01.2022	4.3	No
API Key disclosure	Program 5	Webapplication	21.01.2022	5.0	No
Private Key disclosure	**Program 6**	**Github Repository**	**06.01.2022**	**7.7**	**Yes**
SSH User enumeration	**Program 7**	**Webapplication**	**04.01.2022**	**5.3**	**Yes**
Subdomain takeover	Program 8	Webapplication	07.01.2022	8.2	No
Outdated SSH (6 years)	Program 9	Webapplication	05.01.2022	5.0	No
WordPress user enumeration	Program 9	Webapplication	05.01.2022	6.1	No

Table 4.1: Vulnerabilities discovered by the prototype submitted on HackerOne

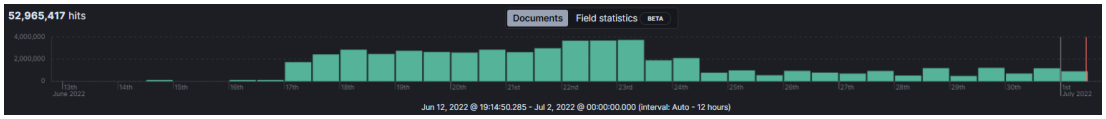

Figure 4.1: Requests of the prototype as registered by the firewall.

Vulnerability	Program	Scope	Reported at	Bugcrowd Rating	Accepted
Logfile Disclosure	**Program 1**	**Webapplication**	**08.06.2022**	**P2**	**Yes**
Disclosed WADL File	Program 1	Webapplication	08.06.2022	P0	No
Reflected XSS	Program 1	Webapplication	08.06.2022	P0	No
Disclosed PHP-Info-File	**Program 1**	**Webapplication**	**08.06.2022**	**P0**	**Yes**
WAF Bypass using WADL Information	Porgram 1	Webapplication	08.06.2022	P0	No
Enabled Directory Listing	**Program 1**	**Webapplication**	**08.06.2022**	**P5**	**Yes**
Reflected XSS	**Program 2**	**Webapplication**	**08.06.2022**	**P3**	**Yes**
Reflected XSS	Program 3	Wepapplication	06.06.2022	P0	No

Table 4.2: Vulnerabilities discovered by the prototype submitted on BugCrowd

Vulnerability	Scope	Reported at	CVSS	Accepted
Server Side Request Forgery	**Webapplication**	**17.06.2022**	**7.5**	**Yes**
Reflected XSS	**Webapplication**	**17.06.2022**	**5.4**	**Yes**
Information Disclosure (Jira)	**Webapplication**	**17.06.2022**	**5.3**	**Yes**
User Enumeration WordPress	**Webapplication**	**17.06.2022**	**3.7**	**Yes**

Table 4.3: Vulnerabilities discovered by the prototype submitted on an Universitys' private program

4.2 Answers to the Research Questions

4.2.1 How precise is an automated Toolchain in the context of Bug Bounty Hunting?

Based on the results in subsection 4.1.2 it can be concluded that the automated toolchain can be generally described as accurate. However, the results from automated approaches must be validated manually since automation may include false positives or detect vulnerabilities which the bug bounty program considers as out of scope. The bug bounty program's rules of engagement must be considered prior to performing automated scans since individual programs may prohibit the use of automated scanners or require a limit based on requests per second. The output of the toolchain lists vulnerabilities which may be valid for the respective bug bounty hunting program must

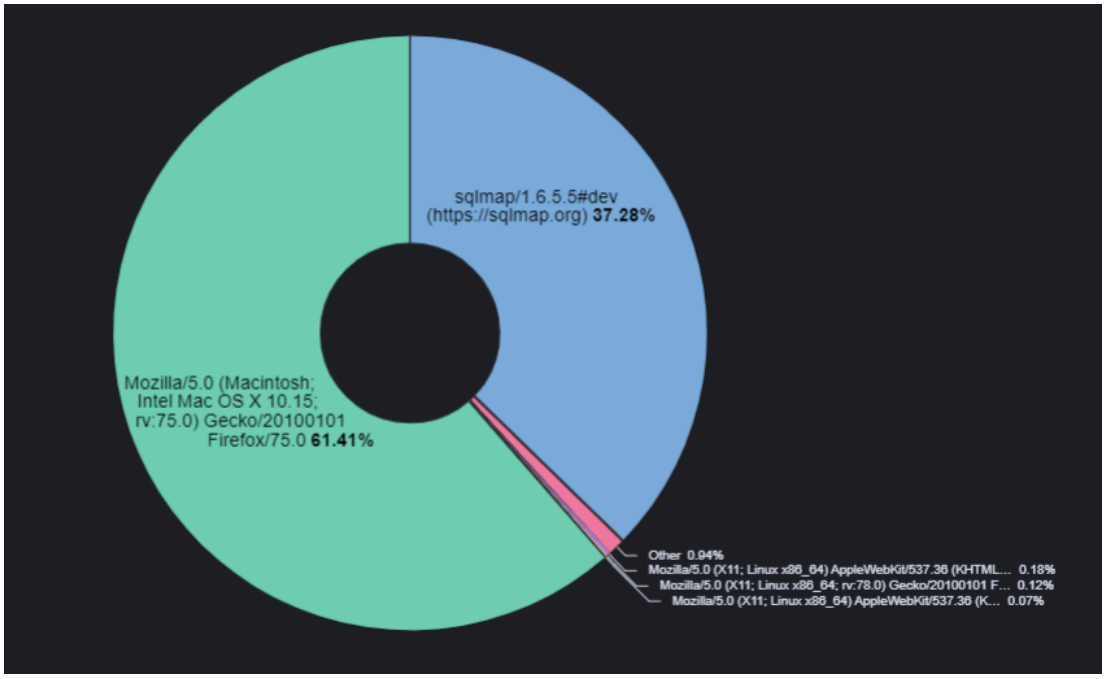

Figure 4.2: Request headers of the prototype as registered by the firewall.

be manually curated and tested since the reports submitted to bug bounty platforms generally have high requirements in terms of impact and plausibility. The raw output of automation should not be submitted directly to the bug bounty platform since the respective vulnerabilities must be explained and demonstrated. However, an automated approach can help to detect vulnerabilities in large scaled bug bounty hunting programs.

4.2.2 Which Types of Vulnerabilities may be Detected?

Based on the coverage of the OWASP Top 10 in section 3.7, eight out of 10 categories are covered by the prototype. The categories not covered are either listed as out of scope by many Bug Bounty Hunting Programs or generally cannot be detected. The category *Insecure design* is only covered partly, since there is often no possibility to check the software design for its security, as source code is often not available for the researcher. However, this category may be covered by employing an automated software security scanner such as SonarQube[2] in context of bug bounty hunting programs aimed towards open source software.

[2]https://www.sonarqube.org/

4.2.3 Can an Automated Toolchain, Composed Mainly of Free Tools Compete in Bug Bounty Hunting?

Based on the results in subsection 4.1.2 it can be concluded that such toolchain can compete in the bug bounty hunting setting. However, automated approaches are becoming more common in bug bounty hunting, which leads to competition among automated approaches. The first researcher utilising automation in a certain bug bounty hunting program generally discovers the most vulnerabilities. Most researchers conducting automated scanning after the first researcher may find duplicated vulnerabilities which have not been yet fixed. These researchers will therefore not be awarded bounties in the most bug bounty hunting programs.

4.2.4 What are the Requirements for the Toolchain in order to qualify for Bug Bounty Hunting Programs?

The requirements differ greatly based on the individual bug bounty hunting program. On many platforms, automated scanners must have a limited amount of requests per second and that a toolchain must thus adapt to these limitations. Many programs require the researcher to send custom headers with each individual request and all tools used in the implementation thus must have the possibility to send custom headers or to be routed over a proxy server which sends these headers. Submitted false positives lead to a degeneration of the reputation of a researcher. All discovered vulnerabilities must therefore be manually curated.

4.2.5 What Cutting-edge Technology has an impact on Bug Bounty Hunting?

Bug bounty programs for IoT devices are on the rise. This development changes the aspect of bug bounty hunting in certain cases. The detection of vulnerabilities for IoT devices differs from the normal setting of bug bounty hunting since these vulnerabilities are often discovered in hardware or vendor-specific software. Normal vulnerability scans may lead to limited results since the approach of vulnerability hunting is different.
Additionally, searching for vulnerabilities on the source code of open source applications is often demanded by certain bug bounty programs. Utilisation of tools to automatically scan source code repositories and analyse the vulnerabilities in the source code can therefore prove to be useful.

4.2.6 What Niches in Bug Bounty Hunting are Not Yet Covered?

It is difficult to conclude which aspects of bug bounty hunting are not yet covered, since not all vulnerability reports are publicly disclosed. Based on the research, vulnerabilities discovered in IoT are described as a niche since few researchers participate in such programs. This is caused by the specifies of the vulnerability discovery, which requires knowledge in both hardware architecture and software development. Additionally, researchers must purchase devices prior to conducting research, which requires a prior financial commitment. Few tools are available to conduct research on IoT devices.

4.3 Outlook

This prototype proved to be a simple implementation of a toolchain which can be used to automatically discover vulnerabilities in web applications as well as source code repositories. However, this prototype is far from an optimal approach to these cases. The prototype must be refined to fulfil its full potential. The architecture of this prototype should be changed to make it less resource intensive. Additionally, the usage of parallelisation should be changed since this prototype is too aggressive for most settings. To avoid this, a rate limiter should be in place and parallelisation should be avoided. To avoid parallelisation, the prototype should be split into multiple parts which execute tools for a certain purpose (e.g. fuzzing) and should be run in docker containers. With this approach, the communication can happen using message queuing and since multiple docker containers can be run at the same time and the implementation itself does not have to rely on parallelisation.
The discovery of vulnerabilities proved to be useful and nuclei proved to be an essential part of the thesis. However, in many occurrences, too many Nuclei templates are run against the target. The Nuclei templates to execute should therefore be mapped to the technology stack of the target. When mapping templates to the technology stack of the target, fewer requests are made to the target since, for example, WordPress templates are not executed on a system which runs Drupal. Based on these improvements, it would be possible to divide the created virtual private server into multiple smaller private servers. Multiple smaller private servers with less resources generally involve lower monthly costs than one large-scale virtual private server.
The prototype currently does not implement rich features for authentication. Vulnerabilities in authenticated sections of the web application are therefore not discovered. To implement rich authentication features, credentials should be stored in a database and the authentication should happen ad hoc. With this approach, more vulnerabilities may be discovered in authenticated parts of web applications.

The database structure of this prototype is also not optimal. It should instead be tailored to collect vulnerabilities and information based on a bug bounty hunting Program rather than on a domain. Each discovered vulnerability should be linked to a domain and each domain should be linked to the bug bounty hunting program. The discovery process is therefore decoupled from a single domain, which allows for the execution of the prototype on a bug bounty hunting program with multiple domains and subdomains in scope, rather than starting the prototype for each domain in the program.

Appendix A

Appendix

A.1 API Documentation

A.1.1 Generic Failures

All methods answer with `500 Internal Server Error` if an error (or an exception within a script) occurred. When `500 Internal Server Error` is received it means that this invocation failed and that no results are produced. Using this technique the client is informed that an error occurred during the execution of the method. Detailed error messages are contained within the server logs.

If a method does not follow this generic failure behaviour it is included in the documentation.

A.1.2 XSSer

Synopsis:	POST `/xsser`
Precondition:	url = string, required, not encoded
	cookie, optional, not encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of vulnerabilities

Example request and response

Request

```
{
     "url": "<url>",
     "cookie": "<cookie>"
}
```

Response

```
{
    "status": 200,
    "vulnerabilities": [
        "[FOUND !!!] -> [ <hash> ] : [ <input name> ]"
    ]
}
```

A.1.3 XSStrike

Synopsis:	POST /xsstrike
Precondition:	url = string, required, not encoded
	cookie, optional, not encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of vulnerabilities

Example request and response

Request

```
{
     "url": "<url>",
     "cookie": "<cookie>"
}
```

Response

```
{
    "status": 200,
    "vulnerabilities": [
        "Testing parameter: <parameterName> \n Reflections found: <n>
    ]
}
```

A.1.4 Sqlmap

Synopsis:	POST /sqlmap
Precondition:	url = string, required, not encoded
	cookie, optional, not encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of vulnerabilities

Example request and response

Request

```
{
     "url": "<url>",
     "cookie": "<cookie>"
}
```

Response

```
{
    "status": 200,
    "vulnerabilities": [
        "<GET/POST> parameter '<parameterName>' is vulnerable."
    ]
```

}

A.1.5 NMAP

Synopsis:	GET `/nmap/:host`
Precondition:	host = string, required, needs to be url encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of ports and an empty state

Example response

Response

```
{
    "status": 200,
    "result": {
    "host": "<host>",
    "state": "<state>",
    "ports": [
        {
            "number": "<number>",
            "state": "<state>",
            "serviceName": "<serviceName>",
            "serviceProduct": "<product>",
            "serviceVersion": "<version>",
            "serviceOs": "<operatingSystem>",
            "vulnerabilities": [
                {
                    "cve": "<CVE-ID>",
                    "cvss": "<CVE-Score>"
                }
                ]
```

```
            }
        ]
    }
}
```

A.1.6 sublist3r

Synopsis:	GET `/sublist3r/:host`
Precondition:	host = string, required, needs to be url encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of domains

Example response

`Response`

```
{
    "status": 200,
    "domains": [
        "<domain1>",
        "<domain2>",
        "<domainN>"
    ]
}
```

A.1.7 Takeover

Synopsis:	POST `/takeover`
Precondition:	domains = array, separate domains are not encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of results

Example request and response

Request

```
{
    "domains": [
        "<domain1>",
        "<domain2>"
    ]
}
```

Response

```
{
    "status": 200,
    "results" : [
        "[ + ] <serviceName> service found! Potential domain takeover
    ]
}
```

A.1.8 Netlas

Synopsis:	POST `/scanCve`
Precondition:	domain = string, required, not encoded
	minimalScore = integer, required
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of vulnerabilities

Example request and response

Request

```
{
    "domain": "domain.com",
    "minimalScore": 1
}
```

Response

```
{
    "status": 200,
    "vulnerabilities": [
        {
            "host": "<domain>",
            "vulnerabilities": [
                {
                    "cveName": "<CVE Number>",
                    "score": "<score>",
                    "severity": "<severity> (e.g.) HIGH",
                    "exploit": "<link to the exploit>"
                }, ...
            ]
        }, ...
    ]
}
```

A.1.9 Availability check

Synopsis:	POST `/checkAvailability`
Precondition:	domains = array, required, separate domains are not encoded
Status code:	200 OK
	500 Internal Server Error
Specifics:	No `417 Expectation Failed` response is sent. If the domain is not reachable `httpSupport` as well as `httpsSupport` are set to `false`

Example request and response

Request

```
{
    "domains": [
    "domain1",
    "domain2",
    ...
    ],
}
```

Response

```
{
    "status": 200,
    "domains": [
        {
            "domain": "domain1",
            "isOnline": true,
            "httpSupport":false,
            "httpsSupport":true
        },
        {
            "domain": "domain2",
            "isOnline": false,
            "httpSupport":false,
```

```
            "httpsSupport":false
        },
        ...
    ]
}
```

A.1.10 Gospider

Synopsis:	POST `/crawl`
Precondition:	url = string, required, not encoded
	cookie = string, optional, not encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of urls and javascript

Example request and response

Request

```
{
    "url": "domain.com",
    "cookie": "<cookie>"
}
```

Response

```
{
    "status": 200,
    "url": "<url>",
    "urls": [
        "<url>/page1",
        "<url>/page2",
         ...
    ],
```

```
    "javascript":[
        "<url>/jquery.js",
        "<url>/client.js"
    ]
}
```

A.1.11 Gowittness

Synopsis:	POST `/screenshot`
Precondition:	url = string, required, not encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of images

Example request and response

Request

```
{
    "url": "domain.com"
}
```

Response

```
{
    "status": 200,
    "domain": "url.com",
    "images": [
        "base64_1",
        "base64_2"
    ]
}
```

A.1.12 Wafw00f

Synopsis:	POST `/wafDetection`
Precondition:	domains = array of string, required, separate domains are not encoded
Status code:	200 OK 500 Internal Server Error
Specifics:	No `417 Expectation Failed` response is sent. If no WAF is detected `hasWaf` is set to false and `manufacturer` as well as `waf` are set to `none`

Example request and response

Request

```
{
    "domains": [
        "domain1",
        "domain2",
        ...
    ],
}
```

Response

```
{
    "status": 200,
    "detections":[
        {
            "domain": "<domain>",
            "hasWaf": true|false,
            "waf": "<WAF> None if no WAF detected",
            "manufacturer": "<Manufacturer> None if no WAF detected"
        }
    ]
}
```

A.1.13 Gobuster

Synopsis:	POST `/directoryScan`
Precondition:	url = string, required, not encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of findings

Example request and response

Request

```
{
    "url": "example.com"
}
```

Response

```
{
    "status": 200,
    "domain": "url.com",
    "findings":[
        {
            "path": "path",
            "status": 200
        },
        ...
    ]
}
```

A.1.14 Nikto

Synopsis:	POST `/scanHost`
Precondition:	url = string, required, not encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of vulnerabilities

Example request and response

Request

```
{
    "domain": "example.com"
}
```

Response

```
{
    "status": 200,
    "domain": "url.com",
    "ip": "ip",
    "banner": "banner",
    "vulnerabilities":[
        {
            "method": "GET",
            "url": "/",
            "finding": "found something!"
        },
        ...
    ]
}
```

A.1.15 Whois lookup

Synopsis:	POST `/whois/lookup`
Precondition:	domain = string, required, not encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` sets all fields to an empty string

Example request and response

Request

```
{
    "domain": "example.com"
}
```

Response

```
{
    "status": 200,
    "domainName":"<domain>",
    "registered":"yes|no",
    "createdAt": "1.1.1900",
    "expiry": "1.1.2999",
    "registrar": {
        "name": "Max",
        "email": "Mustermann",
        "phone": "0664 069420"
    },
    "contact":{
        "name": "Max Mustermann",
        "company": "Muster AG",
        "address": "Musterstraße 1",
        "city": "Mustercity",
        "state": "MusterState",
        "country": "MusterCountry",
        "phone": "0664 069420",
        "email": "max@mustermann.com"
```

```
    }
}
```

A.1.16 Reverse whois lookup

Synopsis:	POST `/whois/reverse`
Precondition:	company = string, required, not encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of vulnerabilities

Example request and response

Request

```
{
    "company": "muster AG"
}
```

Response

```
{
    "status": 200,
    "domains":[
        "domain1",
        ...
    ]
}
```

A.1.17 Retire JS

Synopsis:	POST `/retire`
Precondition:	url = string, required, not encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of vulnerable components

Example request and response

Request

```
{
    "url": "https://example.com/jquery.js"
}
```

Response

```
{
    "status": 200,
    "url": "https://example.com/jquery.js",
    "vulnerableComponents": [
        {
            "cve": "CVE-XXXXXX",
            "description": "this is vulnerable",
            "severity": "9.1"
        },
        ...
    ]
}
```

A.1.18 Nuclei

Synopsis:	POST `/nuclei`
Precondition:	url = string, required, not encoded
	cookie = string, optional, not encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of findings

Example request and response

Request

```
{
    "url": "https://example.com/page1",
    "cookie": "<cookie>"
}
```

Response

```
{
    "status": 200,
    "url": "https://example.com/vulnerable",
    "findings": [
        {
            "template": "<templateThatDetectedTheVulnerability",
            "poc":"<ProofOfConcept>",
            "extractions": [
                "extracted1",
                ...
            ]
        },
        ...
    ]
}
```

A.1.19 Trufflehog

Synopsis:	POST `/git/scan`
Precondition:	repositoryUrl = string, required, not encoded
Status code:	200 OK
	417 Expectation Failed
	500 Internal Server Error
Specifics:	`417 Expectation Failed` returns an empty array of findings

Example request and response

Request

```
{
    "repositoryUrl": "https://github.com/myRepo",
}
```

Response

```
{
    "status": 200,
    "url": "https://github.com/myRepo",
    "findings": [
        {
            "commit": "leakingCommit",
            "commitMessage": "commitMessage",
            "path": "pathOfLeakingFile",
            "diff": "diff to the previous commit",
            "strings":[
                "leakedKey1",
                "leakedPw1",
                ...
            ]
        },
        ...
    ]
```

```
}
```

Acronyms

CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
PoC	Proof of Concept
API	Application Programming Interface
REST	Representational State Transfer
JSON	Javascript Object Notation
RAM	Random Access Memory
npm	Node Package Manager
XSS	Cross Site Scripting
DOM	Document Object Model
SDK	Software Development Kit
HTTP	Hypertext Transfer Protocol
HTTPS	Hyptertext Transfer Protocol Secure
SQLi	SQL injection
SSTI	Server Side Template Injection
DBMS	Database Management System
URL	Uniform Resource Locator
JS	Javascript
CSS	Cascading Style Sheet
WAF	Web Application Firewall
NSE	Nnmap Scripting Engine
TCP	Transmission Control Protocol
UDP	User Datagram Protocol

IDS	Intrusion Detection System
XML	Extensible Markup Language
YAML	Yet Another Markup Language
RCE	Remote Code Execution
HTML	Hypertext Markup Language
CSS	Cascading Style Sheet
IP	Internet Protocol
OWASP	Open Web Application Security Project
CWE	Common Weakness Enumeration
CI/CD	Continuous Integration / Continuous Delivery
ORM	Object Relational Mapping
SQL	Structured Query Language
SSRF	Server Side Request Forgery
IDOR	Insecure Direct Object Reference
CSRF	Cross Site Request Forgery
IoT	Internet of Things
SNMP	Simple Network Management Protocol
SSD	Solid State Drive
CPU	Central Processing Unit
CMS	Content Management System
URLs	Uniform Resource Locators
GB	Gigabyte
ZAP	OWASP Zed Attack Proxy

Bibliography

Akgul, O., T. Eghtesad & A. Laszka (2020). "The Hackers Viewpoint: Exploring Challenges and Benefits of Bug-Bounty Programs". In: (cit. on pp. 1, 5).

Apple (2022). *Apple Security Bounty*. Available from: https://developer.apple.com/security-bounty/ [June 29, 2022] (cit. on p. 11).

Brandon, C. (2022). *PostgreSQL vs. MySQL: what you need to know*. Available from: https://www.fivetran.com/blog/postgresql-vs-mysql#:~:text=PostgreSQL%20is%20an%20object%2Drelational,%2C%20ACID%2Dcompliant%20storage%20engine. [June 12, 2022] (cit. on p. 41).

BugCrowd (2022). *Programs*. Available from: https://bugcrowd.com/programs [July 22, 2022] (cit. on p. 16).

Corfield, G. (2022). *Das tut mir leid! Germany's ruling party sorry for calling cops on researcher after she outed canvassing app flaws*. Available from: https://www.theregister.com/2021/08/05/germany_responsible_disclosure_cdu/ [June 26, 2022] (cit. on p. 10).

Ding, A. Y., G. L. D. Jesus & M. Janssen (2019). "Ethical hacking for boosting IoT vulnerability management: a first look into bug bounty programs and responsible disclosure". In: *Proceedings of the Eighth International Conference on Telecommunications and Remote Sensing* (cit. on pp. 33, 34).

FiRST (2022a). *A Complete Guide to the Common Vulnerability Scoring System*. Available from: https://www.first.org/cvss/v2/ [June 22, 2022] (cit. on p. 24).

– (2022b). *Common Vulnerability Scoring System v3.1: User Guide*. Available from: https://www.first.org/cvss/v3.1/user-guide [June 22, 2022] (cit. on p. 24).

Fryer, H. & E. Simperl (2019). "Web Science Challenges in Researching Bug Bounties". In: (cit. on pp. 5–8).

GitHub (2022). *GitHub Security Bug Bounty*. Available from: https://bounty.github.com/ [June 29, 2022] (cit. on p. 11).

HackerOne (2022a). *Directory: Programs*. Available from: https://hackerone.com/directory/programs [July 1, 2022] (cit. on p. 16).

– (2019). *Submitting Vulnerabilities Increased by 63 percent In 2020*. Available from: https://www.hackerone.com/press-release/number-hackers-submitting-vulnerabilities-increased-63-2020 [Mar. 9, 2019] (cit. on p. 16).

– (2022b). *The HackerOne Top 10 Most Impactful and Rewarded Vulnerability Types – 2020 Edition*. Available from: https://www.hackerone.com/top-ten-vulnerabilities [June 29, 2022] (cit. on pp. 12, 13).

Hakluke (2021). *Hakluke: Creating the Perfect Bug Bounty Automation*. Available from: https://labs.detectify.com/2021/11/30/hakluke-creating-the-perfect-bug-bounty-automation/ [Nov. 30, 2021] (cit. on pp. 2, 19–23).

Kiwi, B. (2021). *Automating bug bounties*. Available from: https://www.benteveo.kiwi/blog/automating-bug-bounties [Feb. 21, 2021] (cit. on pp. 22, 23).

Kuehn, A. & M. Mueller (2014). "Analyzing Bug Bounty Programs: An Institutional Perspective on the Economics of Software Vulnerabilities". In: *2014 TPRC / 42nd Research Conference on Communication, Information and Internet Policy, George Mason University School of Law, Arlington, Virginia, September 12-14, 2014*, pp. 1–13 (cit. on pp. 6, 7, 11).

m8r0wn (2021). *Intro to Bug Bounty Automation: Tool Chaining with Bash*. Available from: https://infosecwriteups.com/intro-to-bug-bounty-automation-tool-chaining-with-bash-13e11348016f [Nov. 30, 2021] (cit. on p. 19).

Malladi, S. & H. C. Subramanian (2020). "Bug Bounty Programs for Cybersecurity: Practices, Issues, and Recommendations". In: *IEEE Software* 37, pp. 31–39 (cit. on p. 19).

Microsoft (2022). *Microsoft Bug Bounty Program*. Available from: https://www.microsoft.com/en-us/msrc/bounty [June 29, 2022] (cit. on pp. 11, 16).

One, H. (2021). *Xiaomi Bug Bounty Program*. Available from: https://hackerone.com/xiaomi?type=team [Feb. 21, 2021] (cit. on p. 35).

OWASP Foundation (2022a). *A01:2021 – Broken Access Control*. Available from: https://owasp.org/Top10/A01_2021-Broken_Access_Control/ [June 22, 2022] (cit. on p. 25).

– (2022b). *A02:2021 – Cryptographic Failures*. Available from: https://owasp.org/Top10/A02_2021-Cryptographic_Failures/ [June 22, 2022] (cit. on pp. 25, 26).

– (2022c). *A03:2021 – Injection*. Available from: https://owasp.org/Top10/A03_2021-Injection/ [June 22, 2022] (cit. on p. 27).

– (2022d). *A04:2021 – Insecure Design*. Available from: https://owasp.org/Top10/A04_2021-Insecure_Design/ [June 22, 2022] (cit. on pp. 27, 28).

– (2022e). *A05:2021 – Security Misconfiguration*. Available from: https://owasp.org/Top10/A05_2021-Security_Misconfiguration/ [June 22, 2022] (cit. on p. 28).

– (2022f). *A06:2021 – Vulnerable and Outdated Components*. Available from: https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/ [June 22, 2022] (cit. on p. 29).

– (2022g). *A07:2021 – Identification and Authentication Failures*. Available from: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/ [June 22, 2022] (cit. on p. 30).

– (2022h). *A08:2021 – Software and Data Integrity Failures*. Available from: https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures/ [June 22, 2022] (cit. on p. 31).

– (2022i). *A09:2021 – Security Logging and Monitoring Failures*. Available from: https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/ [June 22, 2022] (cit. on p. 32).

– (2022j). *A10:2021 – Server-Side Request Forgery (SSRF)*. Available from: https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_%28SSRF%29/ [June 22, 2022] (cit. on p. 33).

– (2022k). *OWASP Top Ten*. Available from: https://owasp.org/www-project-top-ten/#:~:text=The%20OWASP%20Top%2010%20is,step%20towards%20more%20secure%20coding. [June 22, 2022] (cit. on pp. 23, 24, 32).

– (2022l). *Who is the Owasp Foundation?* Available from: https://owasp.org/ [June 22, 2022] (cit. on p. 23).

PayPal (2022). *PayPal Bug Bounty Program*. Available from: https://hackerone.com/paypal?type=team [June 29, 2022] (cit. on p. 11).

Projectdiscovery (2022). *Nuclei Templates*. Available from: https://github.com/projectdiscovery/nuclei-templates [July 18, 2022] (cit. on p. 64).

Shafigh, S., B. Benatallah, C. Rodríguez & M. Al-Banna (2021). "Why Some Bug-bounty Vulnerability Reports are Invalid?: Study of bug-bounty reports and developing an out-of-scope taxonomy model". In: *Proceedings of the 15th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)* (cit. on pp. 13, 14).

Sharma, M. (2013). *Bugcrowd Raises* 1.6*MillionToExpandBugBountyMarketplace*. Available from: https://techcrunch.com/2013/09/04/bugcrowd-raises-1-6-million-to-expand-bug-bounty-marketplace/?guccounter=1 [Sept. 4, 2013] (cit. on p. 16).

Shepherd, S. (2021). "How do we define Responsible Disclosure?" In: *Sans Instutute*, pp. 1–17 (cit. on pp. 9, 10).

Subramanian, H. C. & S. Malladi (2020). "Bug Bounty Marketplaces and Enabling Responsible Vulnerability Disclosure: An Empirical Analysis". In: *J. Database Manag.* 31, pp. 38–63 (cit. on pp. 6, 11, 12, 15, 16).

Sullo (2018). *Overview Description*. Available from: https://github.com/sullo/nikto/wiki/Overview-&-Description [July 18, 2018] (cit. on p. 62).

Tesla (2022). *Tesla's Bug Bounty Program*. Available from: https://bugcrowd.com/tesla [June 29, 2022] (cit. on p. 11).

Union, W. (2022). *Western Union's Bug Bounty Program*. Available from: https://bugcrowd.com/westernunion [June 29, 2022] (cit. on p. 11).

Walshe, T. & A. Simpson (2020). "An Empirical Study of Bug Bounty Programs". In: *2020 IEEE 2nd International Workshop on Intelligent Bug Fixing (IBF)*, pp. 35–44 (cit. on pp. 1, 6, 13).

Yahoo (2022). *Yahoo! Bug Bounty Program*. Available from: https://hackerone.com/yahoo?type=team [June 29, 2022] (cit. on p. 11).

Zhao, M., A. Laszka & J. Grossklags (Feb. 2017). "Devising Effective Policies for Bug-Bounty Platforms and Security Vulnerability Discovery". In: *Journal of Information Policy* 7.1, pp. 372–418. ISSN: 2381-5892. DOI: 10.5325/jinfopoli.7.1.0372. Available from: https://doi.org/10.5325/jinfopoli.7.1.0372 (cit. on pp. 17–19, 35).

Zou, X. (2022). *IoT devices are hard to patch: Here's why—and how to deal with security*. Available from: https://techbeacon.com/security/iot-

devices - are - hard - patch - heres - why - how - deal - security [July 1, 2022] (cit. on pp. 34, 35).